

SOFTWARE ENGINEERING · OBJECT-ORIENTED DESIGN

Design Patterns

A complete study guide to the 23 classic patterns



5	7	11
CREATIONAL	STRUCTURAL	BEHAVIORAL

Every pattern covered with intent, the problem it solves, the solution, structure and runtime diagrams, runnable TypeScript, trade-offs, and real-world uses.

Includes design principles, a decision guide, pattern-relationship maps, common misuses, and a self-test question bank.

Original text written for self-study. Pattern names and the catalog structure follow the classic *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994); all explanations and code here are newly written.

Contents

Part I — Foundations

What a pattern is	vocabulary, anatomy, criticism
Design principles	encapsulate change, interfaces, composition, SOLID
How to read this guide	study plan, notation

Part II — Creational Patterns 5

Factory Method	subclass decides which object to create
Abstract Factory	families of related objects
Builder	step-by-step construction of complex objects
Prototype	create by cloning an existing instance
Singleton	one instance, global access

Part III — Structural Patterns 7

Adapter	make incompatible interfaces work together
Bridge	split abstraction from implementation
Composite	treat trees and leaves uniformly
Decorator	add behaviour by wrapping
Facade	one simple door to a complex subsystem
Flyweight	share state to fit more objects in RAM
Proxy	a stand-in that controls access

Part IV — Behavioral Patterns 11

Chain of Responsibility	pass a request along handlers
Command	turn a request into an object
Iterator	traverse without exposing internals
Mediator	centralise chaotic communication
Memento	snapshot and restore state
Observer	notify subscribers of events
State	behaviour changes with internal state
Strategy	swap interchangeable algorithms
Template Method	skeleton in the base, steps in subclasses
Visitor	new operations without touching classes
Interpreter	a class per grammar rule

Part V — Applying Patterns

Choosing a pattern	symptom → candidate decision table
Look-alike pairs	Strategy vs State, Proxy vs Decorator, and friends
Relationship map	diagram + table of common combinations
Misuse and over-engineering	when not to reach for a pattern
Self-test bank	60 questions with answers

Part I — Foundations

1 • What a design pattern is

A design pattern is a **named, reusable solution to a recurring design problem**. It is not a library you install and not an algorithm you copy. An algorithm prescribes exact steps; a pattern prescribes a *shape* — which responsibilities live in which objects and how those objects reference each other. Two implementations of the same pattern in two codebases can share almost no lines of code and still both be correct instances of it.

Three reasons patterns are worth memorising:

- **Vocabulary.** "Wrap it in a decorator" replaces five minutes of whiteboard. The naming is the single biggest practical benefit.
- **Pre-solved trade-offs.** Each pattern comes with a known cost. You inherit decades of other people's mistakes for free.
- **Recognition.** Most large frameworks are built out of these shapes. Once you can name them you can read unfamiliar code much faster.

ANATOMY OF A PATTERN ENTRY

Every pattern in this guide is described with the same eight fields, so you can compare them directly:

FIELD	WHAT IT TELLS YOU
Intent	One sentence. If you remember only one thing, remember this.
Problem	The concrete pain that makes the pattern necessary. Patterns applied without this pain are over-engineering.
Solution	The restructuring, in prose.
Structure	A diagram of the participating classes and their relationships.
Code	A compact but complete TypeScript example.
Applicability	Signals in your own code that suggest reaching for it.
Trade-offs	Pros and cons side by side. There is always a con.
In the wild	Where you have already used it without knowing.

THE THREE CATEGORIES

CATEGORY	CONCERN	MNEMONIC
Creational	How objects get made	"Who calls <code>new</code> , and how do I stop caring?"
Structural	How objects are composed into bigger structures	"What does this object <i>wrap</i> or <i>contain</i> ?"
Behavioral	How responsibility and communication are assigned	"Who talks to whom, and who decides?"

HONEST CRITICISM

Patterns have a real dark side. Several exist mainly to work around missing language features — in a language with first-class functions, Strategy and Command often collapse to "pass a function"; Iterator is a `for...of` loop; Singleton is a module-level constant. The 1994 catalogue assumed C++ and Smalltalk. Learn the patterns for the vocabulary and the recognition, but always ask whether your language already solves it in one line.

2 • The principles underneath

Every pattern is an application of a handful of deeper principles. If you internalise these you can often derive the right pattern rather than recall it.

2.1 Encapsulate what varies

Find the part of your system most likely to change, and isolate it behind a stable boundary so that a change there does not ripple outward. This is the root of nearly the whole catalogue: Strategy encapsulates a varying algorithm, Factory Method a varying class choice, State a varying mode of behaviour, Decorator a varying stack of extras.

2.2 Program to an interface, not an implementation

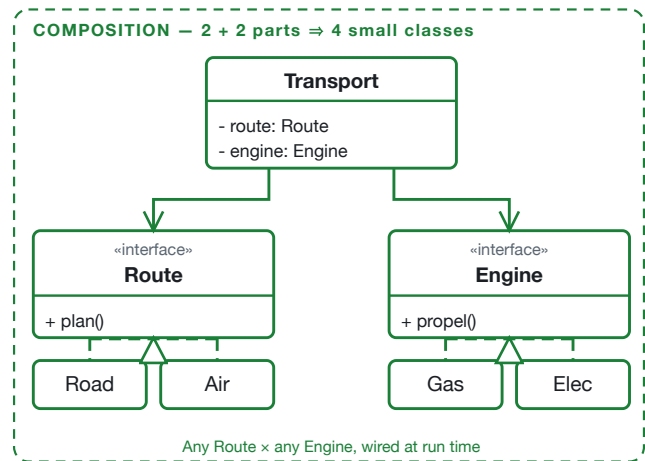
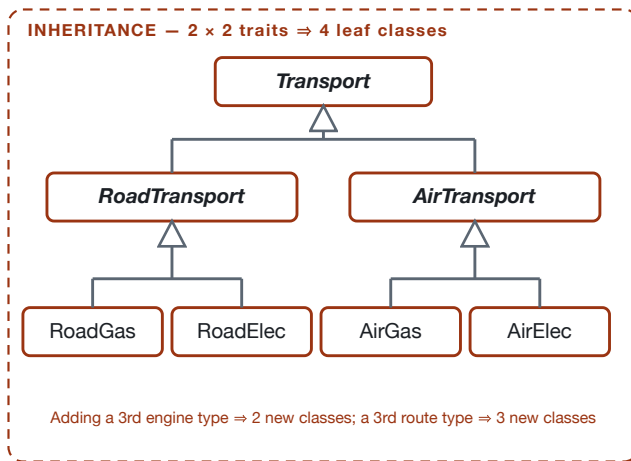
Depend on the narrowest contract that gets your job done. Then any object satisfying that contract — including test doubles and implementations that do not exist yet — can be substituted.

```
// Coupled: Report can only ever produce PDFs.
class Report { save() { new PdfWriter().write(this.body) } }

// Decoupled: Report works with anything that can write.
interface Writer { write(text: string): void }
class Report {
  constructor(private writer: Writer) {}
  save() { this.writer.write(this.body) }
}
```

2.3 Favour composition over inheritance

Inheritance is the tightest coupling in object-oriented programming: a subclass depends on its parent's internals, cannot change parent at run time, and every combination of behaviours needs its own class. Composition — holding a reference to a collaborator — is looser, swappable at run time, and combines linearly instead of exponentially.



Inheritance multiplies classes; composition adds parts and defers the combination to run time.

2.4 SOLID

LETTER	PRINCIPLE	IN PRACTICE
S	Single Responsibility	A class should have one reason to change. If you describe it with "and", split it.
O	Open/Closed	Open to extension, closed to modification. Add a new subclass or plug in a new strategy instead of editing a working class.
L	Liskov Substitution	A subtype must be usable anywhere its supertype is: don't strengthen preconditions, weaken postconditions, or throw new exception types.
I	Interface Segregation	Many small interfaces beat one fat one. Clients shouldn't be forced to depend on methods they never call.
D	Dependency Inversion	High-level policy shouldn't depend on low-level detail; both should depend on an abstraction. This is what makes dependency injection work.

LISKOV, CONCRETELY

The classic violation: `Square` extends `Rectangle`. Any code doing `r.setWidth(5); r.setHeight(4); assert(r.area() == 20)` breaks when handed a `Square`. The subclass satisfied the compiler and violated the contract.

3 · How to use this guide

A FOUR-WEEK PLAN

WEEK	READ	DO
1	Part I, then Strategy, Observer, Decorator, Adapter, Factory Method	These five cover ~70% of real-world use. Find each one in a framework you already use.
2	Remaining creational + structural	Implement Builder and Composite from scratch, no reference.
3	Remaining behavioral	Refactor one real switch-heavy class of yours into State or Command.
4	Part V	Work the 60-question bank. Anything you miss, re-read that entry only.

DIAGRAM NOTATION

NOTATION	MEANING
<i>Italic name</i>	Abstract class, or abstract method a subclass must implement
«interface»	Interface (pure contract, no state)
Solid line, hollow triangle ▷	Extends (class inheritance)
Dashed line, hollow triangle ▷	Implements (interface realisation)
Solid line, open arrow →	Holds a reference to (association); a hollow diamond ◇ at the source marks a container of children
Dashed line, open arrow →	Uses or creates temporarily (dependency); labelled «create» when it constructs
+ / # / -	Public / protected / private member; <u>underlined</u> = static
Yellow folded note	Explanatory note or code excerpt attached to an element
Sequence diagram	Solid arrow = call, dashed arrow = return; a small loop = a call to itself; amber box = commentary on that moment

Diagrams are colour-coded by category (blue creational, purple structural, green behavioral). Every pattern has a **Structure** diagram (classes and relationships); most also have a **Runtime** view — a sequence, object, or state diagram of the code example actually executing.

Code is TypeScript, chosen because interfaces are explicit and the syntax reads close to Java, C#, and Swift. Examples are trimmed to the pattern — imports, error handling, and validation are omitted deliberately.

Part II — Creational Patterns

Creational patterns all answer one question: **how does an object get made, and who is allowed to know its concrete class?** The direct `new Concrete()` call is the tightest possible coupling — it hard-codes a class name into a caller. Each pattern below buys back flexibility at a different price.

PATTERN	VARIES	COST
Factory Method	Which single class to instantiate	A subclass per variant
Abstract Factory	Which whole <i>family</i> of classes	Adding a product type touches every factory
Builder	The construction <i>steps</i> and their order	An extra builder class
Prototype	Nothing — you copy a configured instance	Deep-copy correctness
Singleton	Nothing — you restrict to one instance	Global state, hostile to tests

Factory Method

also: *Virtual Constructor*

INTENT

Define an interface for creating an object, but let subclasses decide which class to instantiate — so a class can defer instantiation to its subclasses.

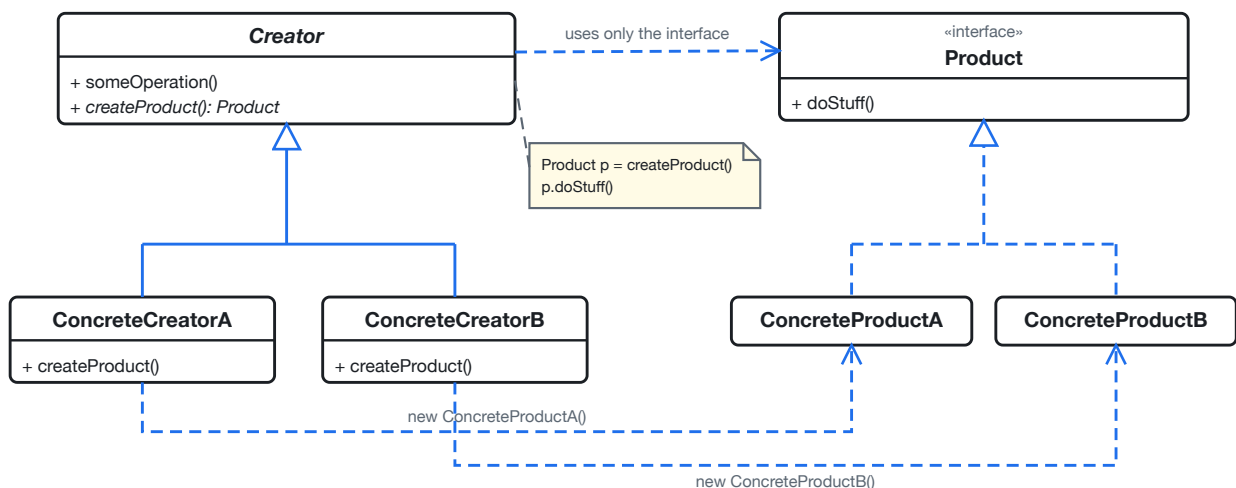
PROBLEM

You wrote a logistics app that only moved goods by truck. `RoadLogistics` is littered with `new Truck()`. Now sea freight arrives. If you add `if (byShip) new Ship() else new Truck()`, you have a conditional that must be repeated at every creation site and extended for every future transport — rail, drone, courier. The class is *closed* to extension in the wrong way: you cannot add a transport without editing existing, working, tested code.

SOLUTION

Replace every direct constructor call with a call to a **factory method** — an overridable method whose only job is to return a product. The base class contains the business logic and calls `this.createTransport()`; each subclass overrides only that one method. The base class never learns the concrete product's name; it only knows the product *interface*.

The crucial insight: the base class's business logic works entirely through the product interface, so it keeps working for products that did not exist when it was written.



Structure. The italic `createProduct()` is the factory method; `someOperation()` calls it and never names a concrete class.

CODE

```
interface Transport { deliver(cargo: string): string }

class Truck implements Transport {
  deliver(c: string) { return `${c} in a box, by road` }
}

class Ship implements Transport {
  deliver(c: string) { return `${c} in a container, by sea` }
}

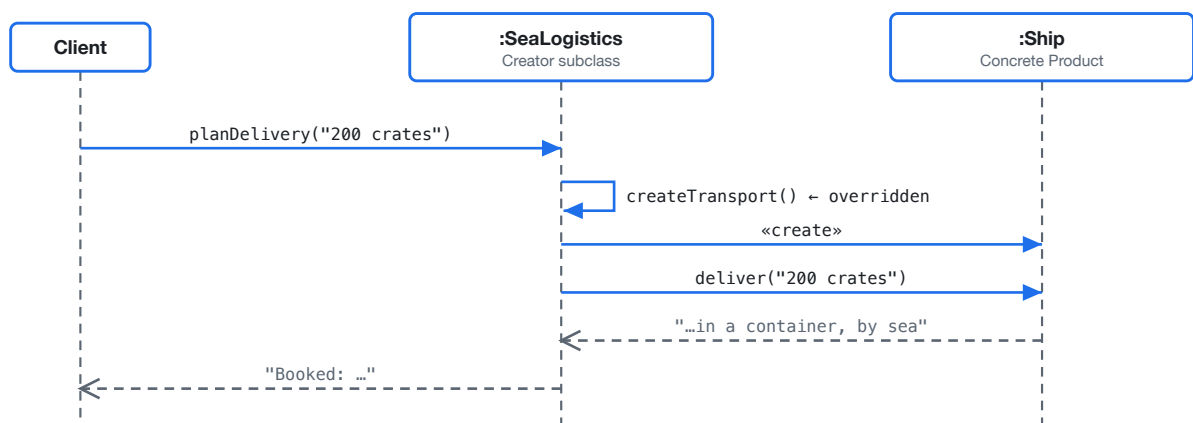
abstract class Logistics {
  // The factory method. Subclasses fill this in.
  protected abstract createTransport(): Transport

  // Business logic – written once, works with any Transport.
  planDelivery(cargo: string): string {
    const t = this.createTransport()
    return "Booked: " + t.deliver(cargo)
  }
}

class RoadLogistics extends Logistics {
  protected createTransport() { return new Truck() }
}

class SeaLogistics extends Logistics {
  protected createTransport() { return new Ship() }
}

// Client code depends only on Logistics.
function run(l: Logistics) { console.log(l.planDelivery("200 crates")) }
run(new SeaLogistics()) // Booked: 200 crates in a container, by sea
```



Runtime. The base-class algorithm runs in the subclass; only the hook decides the concrete type.

VARIATION: PARAMETERISED FACTORY

A single method with a switch over a type key (`create(kind)`) is often called a factory method too, and is sometimes labelled *Simple Factory*. It is not the GoF pattern — there is no subclassing and no open/closed benefit — but it is frequently the right pragmatic choice when the set of products is small and closed.

APPLICABILITY

- You don't know in advance the exact types your code should work with.
- You want users of a framework to extend its internal components (a UI framework's `createButton()`).

- You want to reuse expensive objects instead of rebuilding them — the factory method can hand back a pooled instance rather than a fresh one, invisibly to callers.

PROS

- Removes concrete class names from business logic.
- Creation code lives in one place per variant.
- New products need no edits to existing code (open/closed).

CONS

- Class count grows: one creator subclass per product.
- Requires an existing hierarchy to hook into, or you invent one just for this.
- Overkill when the product set never changes.

IN THE WILD

- Framework hook methods: Android's `Fragment.onCreateView()`, Java's `URLStreamHandler.openConnection()`, Spring's `FactoryBean.getObject()`.
- Java's `Collection.iterator()` — each collection subclass returns its own iterator type; the caller only sees `Iterator`.
- Logger libraries: `LoggerFactory.getLogger()` returning an implementation chosen by configuration.

Relations: Often the starting point that evolves into **Abstract Factory**, **Prototype**, or **Builder**. **Template Method** is its behavioral twin — a factory method is literally a template method whose product is an object.

Abstract Factory

also: Kit

INTENT

Provide an interface for creating *families* of related objects without naming their concrete classes — and guarantee the members you get are compatible with each other.

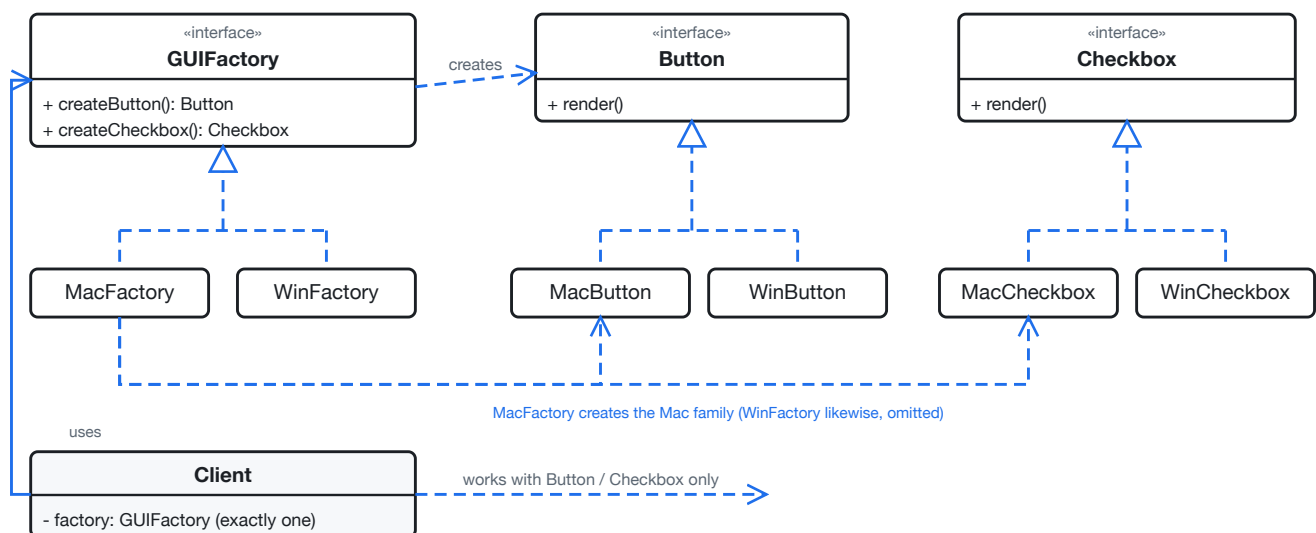
PROBLEM

You build cross-platform UI widgets. You have two independent axes: the widget kind (`Button` , `Checkbox` , `Scrollbar`) and the platform (`macOS` , `Windows` , `Web`). That is nine concrete classes, and the real danger isn't the count — it's **mismatch**. Nothing stops a caller from combining a `MacButton` with a `WinCheckbox` and producing an incoherent UI.

SOLUTION

Declare an interface for each product kind. Then declare a single **abstract factory** interface with one creation method per product kind. Each concrete factory implements the whole set for one family. Because a caller holds exactly one factory, every object it produces necessarily belongs to the same family — the mismatch becomes unrepresentable.

Choose the concrete factory once, at start-up, from configuration or environment detection. The rest of the application never sees a platform name again.



Structure. Each concrete factory produces one family; the client holds a single factory, so mixing families is unrepresentable.

CODE

```
// --- product interfaces (one per kind) ---
interface Button { render(): string }
interface Checkbox { render(): string }

// --- family: macOS ---
class MacButton implements Button { render() { return "[ rounded button ]" } }
class MacCheckbox implements Checkbox { render() { return "( ✓ )" } }

// --- family: Windows ---
class WinButton implements Button { render() { return "< square button >" } }
class WinCheckbox implements Checkbox { render() { return "[x]" } }

// --- the abstract factory ---
interface GUIFactory {
  createButton(): Button
  createCheckbox(): Checkbox
}
class MacFactory implements GUIFactory {
  createButton() { return new MacButton() }
  createCheckbox() { return new MacCheckbox() }
}
class WinFactory implements GUIFactory {
  createButton() { return new WinButton() }
  createCheckbox() { return new WinCheckbox() }
}

// --- client: knows no concrete class ---
class SettingsDialog {
  private ok: Button; private remember: Checkbox
  constructor(f: GUIFactory) {
    this.ok = f.createButton()
    this.remember = f.createCheckbox()
  }
  draw() { return this.remember.render() + " " + this.ok.render() }
}

// Chosen exactly once, at the entry point:
const factory = process.platform === "darwin" ? new MacFactory() : new WinFactory()
console.log(new SettingsDialog(factory).draw())
```

FACTORY METHOD VS ABSTRACT FACTORY

	FACTORY METHOD	ABSTRACT FACTORY
Products created	One	Several related ones
Mechanism	Inheritance — override a method	Composition — hold a factory object
Adding a variant	Cheap: one new subclass	Cheap: one new factory
Adding a product kind	N/A	Expensive: change the factory interface and every implementation

PROS

- Guarantees products from one family are used together.
- Swapping the whole family is a one-line change at start-up.
- Creation code fully isolated from use.

CONS

- Adding a new product *kind* breaks every factory — the pattern is closed on the wrong axis.
- Substantial boilerplate for a two-family problem.
- Easy to introduce "just in case" before a second family exists.

In the wild: `javax.xml.parsers.DocumentBuilderFactory`; Swing's pluggable look-and-feel (`UIManager` / `LookAndFeel`); .NET's `DbProviderFactory` (connection + command + adapter for one database); database dialect providers in ORMs.

Relations: Often begins life as a set of **Factory Methods**; concrete factories are frequently **Singletons**; can be implemented with **Prototype** by cloning stored exemplars instead of instantiating.

Builder

INTENT

Separate the construction of a complex object from its representation, so the same construction process can produce different results — and so callers never face a constructor with twelve optional parameters.

PROBLEM

A `House` may or may not have a garage, pool, garden, heating, or a fourth floor. Two bad answers present themselves:

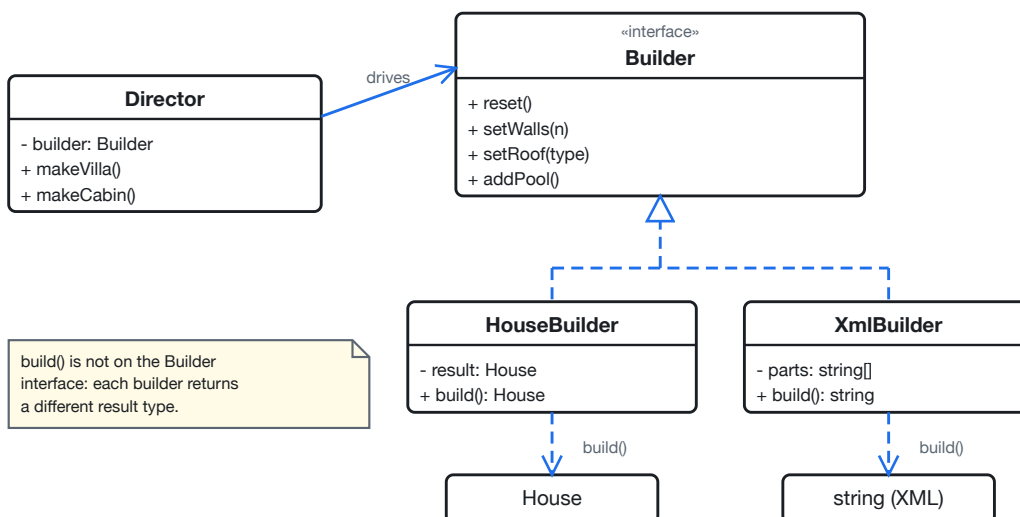
- **Telescoping constructors.** `new House(4, true, false, null, true, 2, false)` — unreadable at the call site, and every new option means a new overload.
- **A subclass per combination.** `HouseWithPoolAndGarage` — exponential blow-up.

A third variant of the problem: you want the *same* construction sequence to produce different outputs — a real object, or an XML description of one, or a SQL insert.

SOLUTION

Move construction into a separate **builder** object exposing one method per optional step. Each step method mutates the builder's work-in-progress and returns `this`, so calls chain readably. A final `build()` hands back the finished product and resets the builder.

Optionally add a **director** that knows useful step sequences (`buildMinimalHouse`, `buildLuxuryVilla`) so recipes are reusable. The director drives any builder, which is where the "same process, different representation" power comes from.



Structure. The director knows recipes; each builder knows one representation.

CODE

```
class House {
  walls = 0; roof = "none"; pool = false; garage = false
  describe() {
    return `${this.walls}-wall house, ${this.roof} roof` +
      (this.pool ? ", pool" : "") + (this.garage ? ", garage" : "")
  }
}

interface Builder {
  reset(): this
  setWalls(n: number): this
  setRoof(t: string): this
  addPool(): this
  addGarage(): this
}

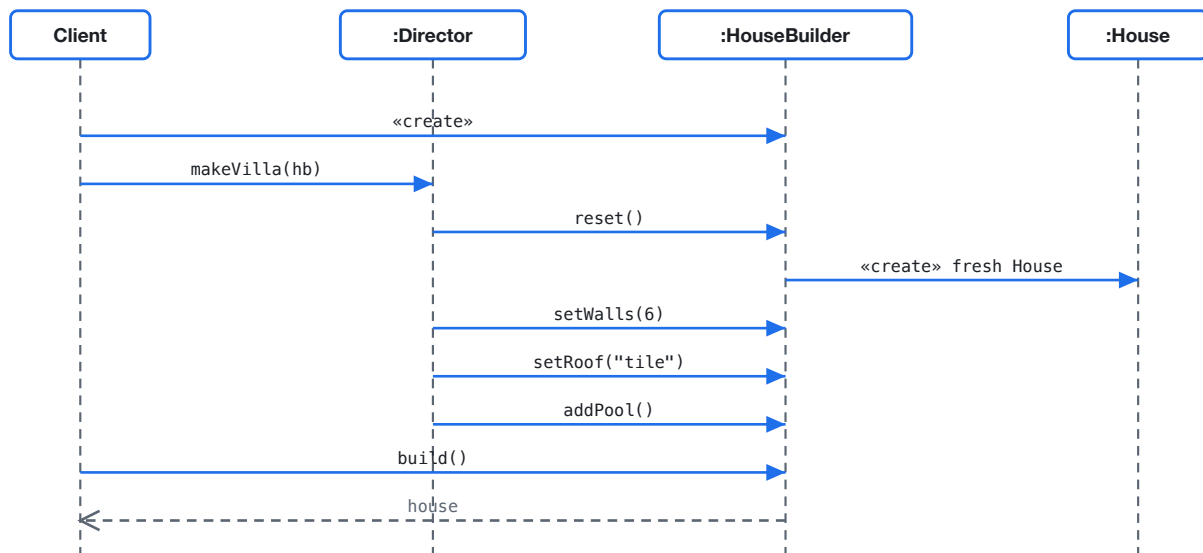
class HouseBuilder implements Builder {
  private h = new House()
  reset() { this.h = new House(); return this }
  setWalls(n: number) { this.h.walls = n; return this }
  setRoof(t: string) { this.h.roof = t; return this }
  addPool() { this.h.pool = true; return this }
  addGarage() { this.h.garage = true; return this }
  // build() is deliberately NOT on the interface: each builder
  // returns a different type, so the director never calls it.
  build(): House { const out = this.h; this.reset(); return out }
}

class XmlBuilder implements Builder {
  private parts: string[] = []
  reset() { this.parts = []; return this }
  setWalls(n: number) { this.parts.push(`<walls>${n}</walls>`); return this }
  setRoof(t: string) { this.parts.push(`<roof>${t}</roof>`); return this }
  addPool() { this.parts.push("<pool/>"); return this }
  addGarage() { this.parts.push("<garage/>"); return this }
  build(): string { const s = `<house>${this.parts.join("")}</house>`; this.reset(); return s }
}

// The director stores reusable recipes.
class Director {
  makeVilla(b: Builder) { b.reset().setWalls(6).setRoof("tile").addPool().addGarage() }
  makeCabin(b: Builder) { b.reset().setWalls(4).setRoof("thatch") }
}

const d = new Director()
const hb = new HouseBuilder(); d.makeVilla(hb)
console.log(hb.build().describe()) // 6-wall house, tile roof, pool, garage

const xb = new XmlBuilder(); d.makeVilla(xb) // same recipe...
console.log(xb.build()) // ...different representation
```



Runtime. The director never calls `build()` — the client does, because only it knows which builder (and result type) it chose.

FLUENT BUILDER ≠ GOF BUILDER

Most "builders" you meet in modern code (`StringBuilder`, query builders, fetch request builders) are the *fluent-interface* half of the pattern only: one builder, one product, no director. That is a perfectly good use — just be aware the "same process, different representation" capability is the part people usually drop.

PROS

- Constructs objects step by step; steps can be skipped or reordered.
- Recipes are reusable across product representations.
- Keeps a bloated constructor out of the product class (SRP).
- Product can stay immutable — it's only assembled once, in `build()`.

CONS

- Extra classes; overkill for objects with two fields.
- Half-built state is representable — `build()` should validate.
- Named/optional parameters or object literals cover many cases in modern languages.

In the wild: `StringBuilder`; SQL/ORM query builders; Protocol Buffers and Lombok-generated `@Builder` classes; `HttpRequest.newBuilder()` in Java; `UriComponentsBuilder` in Spring. **Relations:** Often introduced when a **Factory Method** grows too many parameters. A **Composite** tree is a classic thing to assemble with a Builder.

Prototype

also: Clone

INTENT

Create new objects by copying an existing, already-configured instance, so the client never needs to know the object's concrete class or how to configure it.

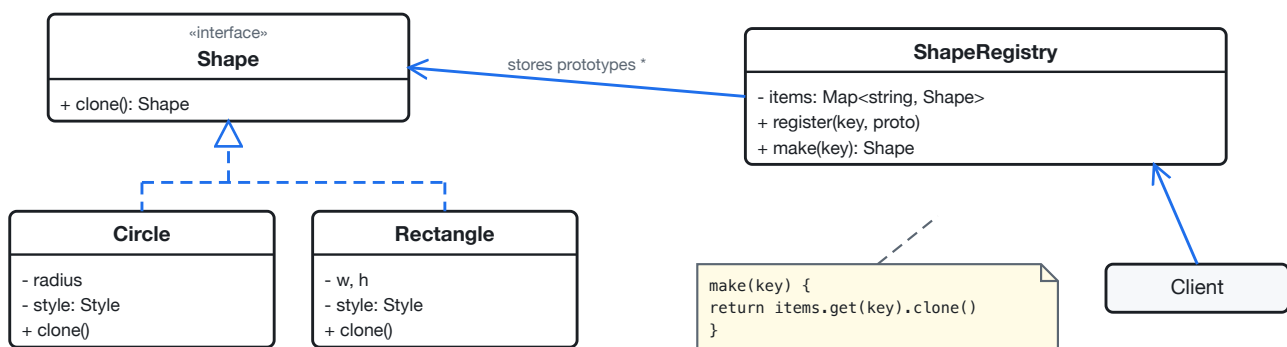
PROBLEM

You have an object and want an exact duplicate. Copying field-by-field from outside fails for two reasons: some fields are private and invisible to you, and the copying code must name the concrete class — coupling you to it. Worse, the object may be expensive to build from scratch (it loaded configuration, opened a connection, parsed a file) when a copy of an existing one would do.

SOLUTION

Put the copying responsibility *inside* the object. Declare a common `clone()` method; each class implements it by constructing a new instance of itself and transferring its own fields — private ones included, since it is inside the class.

The decisive question in every implementation is **shallow or deep**. A shallow clone copies references, so the original and copy share nested objects and mutating one is visible in the other. A deep clone recursively copies them. Neither is universally right: shared immutable data should be shallow-copied; mutable owned data must be deep-copied.



Structure. A registry of configured prototypes replaces both `'new'` and a subclass per configuration.

CODE

```
interface Shape {
  clone(): Shape
  area(): number
}

class Style { constructor(public colour: string, public width: number) {} }

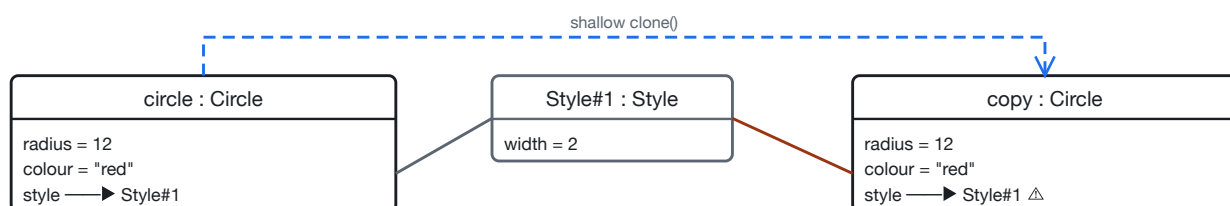
class Circle implements Shape {
  constructor(private r: number, private style: Style) {}

  clone(): Circle {
    // DEEP: the style belongs to this circle, so copy it too.
    return new Circle(this.r, new Style(this.style.colour, this.style.width))
  }
  area() { return Math.PI * this.r ** 2 }
  recolour(c: string) { this.style.colour = c }
  colour() { return this.style.colour }
}

// A registry of named prototypes – the pattern's most useful form.
class ShapeRegistry {
  private items = new Map<string, Shape>()
  register(key: string, proto: Shape) { this.items.set(key, proto) }
  make(key: string): Shape {
    const p = this.items.get(key)
    if (!p) throw new Error(`no prototype: ${key}`)
    return p.clone() // no `new`, no class name, no switch
  }
}

const reg = new ShapeRegistry()
reg.register("warning-dot", new Circle(4, new Style("amber", 1)))

const a = reg.make("warning-dot") as Circle
const b = reg.make("warning-dot") as Circle
a.recolour("red")
console.log(a.colour(), b.colour()) // red amber ← deep copy worked
```



Shallow copy: both circles now share Style#1 — recolouring one recolours the other. Deep copy duplicates Style#1.

Object diagram. The shallow/deep decision is the whole difficulty of the pattern.

TRAPS

CYCLES. A naive recursive deep clone infinite-loops on `a.b = b`; `b.a = a`. Carry a `Map<original, copy>` and return the existing copy when you revisit a node.

SERIALIZATION SHORTCUTS. `JSON.parse(JSON.stringify(x))` silently destroys `Date`, `Map`, `Set`, functions, `undefined`, and class identity. Use `structuredClone()` in modern JS, or write `clone()` by hand.

IDENTITY FIELDS. A clone usually needs a *new* id, timestamp, and version — copying those verbatim is a common bug.

PROS

- Clone without coupling to concrete classes.
- Skips expensive initialisation.
- Replaces subclass-per-configuration with data (registered prototypes).
- Objects can be added to the registry at run time.

CONS

- Deep-cloning graphs with cycles and shared refs is genuinely tricky.
- Every class must implement `clone()` — easy to forget on a new field.
- `final` / `readonly` fields can make copy constructors awkward.

In the wild: JavaScript's own object model is prototypal; `Object.create`, `structuredClone`, Java's `Cloneable`, C#'s `ICloneable` / `MemberwiseClone`; game engines spawning entities from templates; undo systems (with **Memento**). **Relations:** An **Abstract Factory** can be built from prototypes. **Memento** is sometimes implemented by cloning. **Composite** and **Decorator** structures benefit from recursive cloning.

Singleton

INTENT

Ensure a class has exactly one instance, and provide a global point of access to it.

PROBLEM

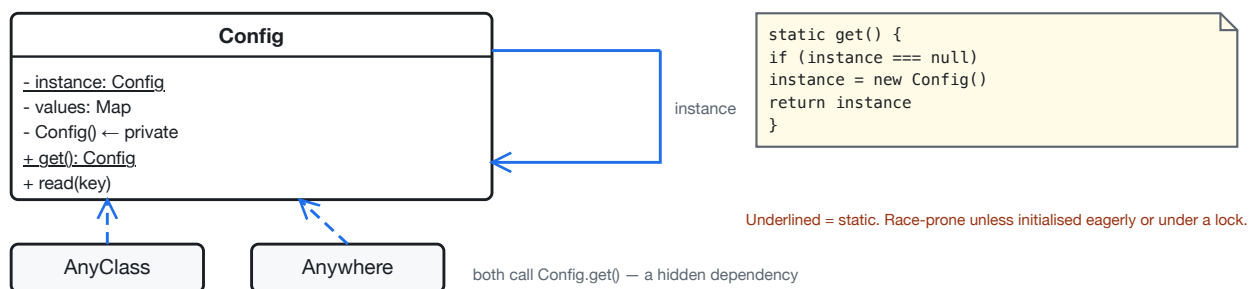
Two needs are bundled into one pattern, which is the root of most arguments about it:

1. **Control the number of instances.** Usually because the object owns a shared resource — a database connection pool, a hardware device, a file lock — and a second instance would be incorrect, not merely wasteful.
2. **Provide global access.** Reach the object from anywhere without threading it through every constructor.

Need 1 is legitimate and occasionally unavoidable. Need 2 is a global variable wearing a costume.

SOLUTION

Make the constructor private so nobody can call `new`. Expose a static accessor that creates the instance on first call, caches it, and returns the cache thereafter.



Structure. A private constructor plus a static accessor holding the one instance. Every caller reaches it globally — which is exactly the problem.

CODE

```

class Config {
  private static instance: Config | null = null
  private values = new Map<string, string>()

  private constructor() { // ← nobody outside can call this
    this.values.set("env", "prod") // expensive load happens once
  }

  static get(): Config {
    if (Config.instance === null) Config.instance = new Config()
    return Config.instance
  }

  read(k: string) { return this.values.get(k) }
}

console.log(Config.get() === Config.get()) // true

```

THREAD SAFETY

The lazy check above is a race in any multi-threaded language: two threads can both see `null` and both construct. Fixes, best first:

1. **EAGER STATIC INITIALISATION.** `private static final Config INSTANCE = new Config();` — the language guarantees it runs once. Choose this unless construction is genuinely too expensive to do at load time.
2. **INITIALISATION-ON-DEMAND HOLDER.** A private static nested class holding the instance; the JVM class-loader gives you lazy *and* safe with no locking.
3. **DOUBLE-CHECKED LOCKING WITH A `VOLATILE` FIELD.** Correct but easy to get wrong; without `volatile` it is broken.
4. **AN ENUM WITH A SINGLE CONSTANT** (Java) — also serialization- and reflection-proof.

WHY IT IS WIDELY CONSIDERED AN ANTI-PATTERN

OBJECTION	DETAIL
Hidden dependencies	<code>Config.get()</code> buried in a method body means the class's real dependencies aren't in its signature. You cannot tell what a class needs by reading its constructor.
Hostile to tests	Global mutable state leaks between tests, making them order-dependent. Substituting a fake requires a reset hook or bytecode tricks.
Violates SRP	The class manages both its own business logic and its own lifecycle.
Wrong assumption	"Exactly one" is stable until it isn't: a second tenant, a second database, a second window, a test needing isolation.

WHAT TO DO INSTEAD

Separate the two needs. Keep the class an ordinary class with a normal constructor and explicit dependencies. Then create exactly one instance at your application's entry point and inject it where needed. You get single-instance behaviour, testability, and honest signatures. Every dependency injection container is essentially "Singleton done properly" — it manages scoped lifetimes without hard-coding global access into the class.

```
// Preferred: a plain class + one instance at the composition root.
class Config { constructor(private values: Map<string,string>) {} /* ... */ }

// main.ts – the only place that decides "one"
const config = new Config(loadEnv())
const app = new App(config, new Db(config)) // dependencies visible

// In JS/TS a module is already a lazy singleton – no pattern needed:
// config.ts → export const config = new Config(loadEnv())
```

PROS

- Genuinely enforces one instance where correctness demands it.
- Lazy: the instance is built only if used.
- Single, obvious access point.

CONS

- Global mutable state, with all its coupling.
- Needs care to be thread-safe.
- Makes unit tests order-dependent and hard to isolate.
- Hides dependencies from a class's public contract.

Legitimate uses: a process-wide logger sink, a connection pool, a hardware driver handle, an interned-value cache, a runtime's scheduler. **Relations:** **Abstract Factory**, **Builder**, and **Prototype** registries are often implemented as singletons. A **Facade** frequently is one too. **Flyweight** resembles it — but a flyweight factory holds *many* shared immutable objects, not one.

Part III — Structural Patterns

Structural patterns are about **composition**: how you assemble objects into larger structures while keeping them flexible. Four of the seven — Adapter, Decorator, Facade, Proxy — are *wrappers*: an object that holds another object and forwards calls to it. They differ only in **why** they wrap. Learning that distinction is most of learning this chapter.

WRAPPER	INTERFACE IT EXPOSES	REASON FOR EXISTING
Adapter	<i>Different</i> from the wrappee's	Translate an incompatible interface
Decorator	<i>Same</i> as the wrappee's	Add behaviour; stackable
Proxy	<i>Same</i> as the wrappee's	Control access — lazily, remotely, with logging or caching
Facade	<i>New and simpler</i>	Hide a whole subsystem behind one door

The remaining three are not wrappers: **Bridge** splits one hierarchy into two, **Composite** builds recursive trees, and **Flyweight** deduplicates memory.

Adapter

also: Wrapper

INTENT

Convert the interface of an existing class into the interface a client expects, letting classes work together that otherwise could not.

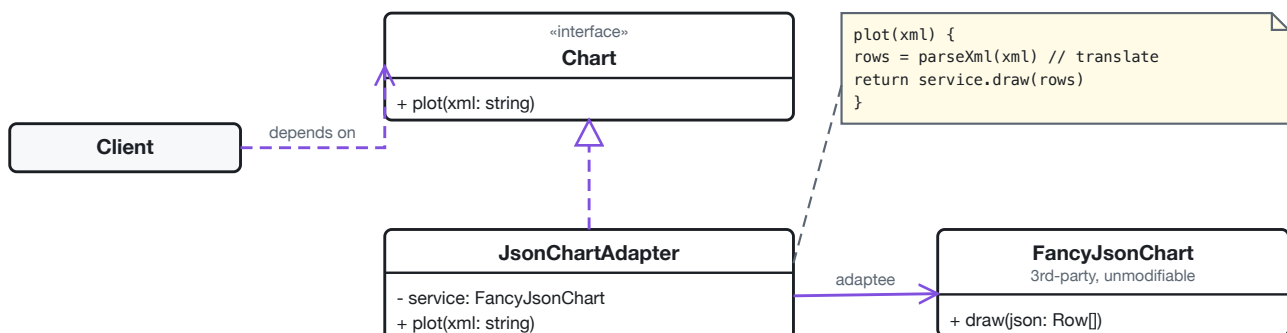
PROBLEM

Your analytics code works with XML. You buy a charting library that only accepts JSON. You cannot edit the library (third-party, possibly closed source), and rewriting your whole app to speak JSON is disproportionate. The two components are individually correct and mutually unusable.

SOLUTION

Introduce an **adapter**: an object that implements the interface your client already speaks, holds a reference to the incompatible service, and translates each call — converting arguments, renaming methods, adjusting units, reshaping return values. The client and the service both remain untouched and unaware of each other.

Two implementations exist. **Object adapter** (shown here) uses composition and is almost always the right choice. **Class adapter** inherits from both the target interface and the adaptee at once; it needs multiple inheritance, so it is a C++ idiom.



Structure (object adapter). The adapter implements the interface the client speaks and holds the incompatible service.

CODE

```
// What our application already speaks.
interface Chart { plot(xml: string): string }

// The third-party library. We cannot change this.
class FancyJsonChart {
    draw(json: { label: string; value: number }[]): string {
        return json.map(d => `${d.label}: ${"█".repeat(d.value)}\n`).join("\n")
    }
}

// The adapter: implements Chart, delegates to FancyJsonChart.
class JsonChartAdapter implements Chart {
    constructor(private service: FancyJsonChart) {}

    plot(xml: string): string {
        const rows = [...xml.matchAll(/<row l="(.*?)" v="(\d+)"\>/g)]
            .map(m => ({ label: m[1], value: Number(m[2]) }))
        return this.service.draw(rows) // ← the translation
    }
}

// Client code is written once, against Chart only.
function report(c: Chart, data: string) { console.log(c.plot(data)) }

report(new JsonChartAdapter(new FancyJsonChart()),
    '<row l="Mon" v="4"><row l="Tue" v="7">')
```

APPLICABILITY

- You want to use an existing class whose interface doesn't match.
- You want to reuse several subclasses that each lack a common capability, and subclassing them all is impractical.
- You are isolating a third-party dependency so it can be swapped later — the adapter is the seam.
- Legacy migration: new code speaks the new interface; adapters keep old implementations alive behind it.

PROS

- Separates conversion from business logic (SRP).
- New adapters need no client changes (open/closed).
- Creates a natural boundary against third-party churn.

CONS

- More indirection to read through.
- Sometimes simpler to change the service class, if you own it.
- Leaky translations (units, nullability, error semantics) are a real bug source.

In the wild: `Arrays.asList()` (array → `List`); `InputStreamReader` (byte stream → character `Reader`); ORM row-to-entity mappers; every "driver" or "provider" layer; SLF4J's binding to a concrete logging backend. **Relations:** **Bridge** is designed in up front, whereas **Adapter** is retrofitted onto code you didn't control. **Decorator** keeps the interface identical; **Adapter** changes it. **Facade** defines a new simpler interface over many objects; **Adapter** re-shapes one.

Bridge

INTENT

Decouple an abstraction from its implementation so the two can vary independently — turning a multiplying class hierarchy into two hierarchies joined by a reference.

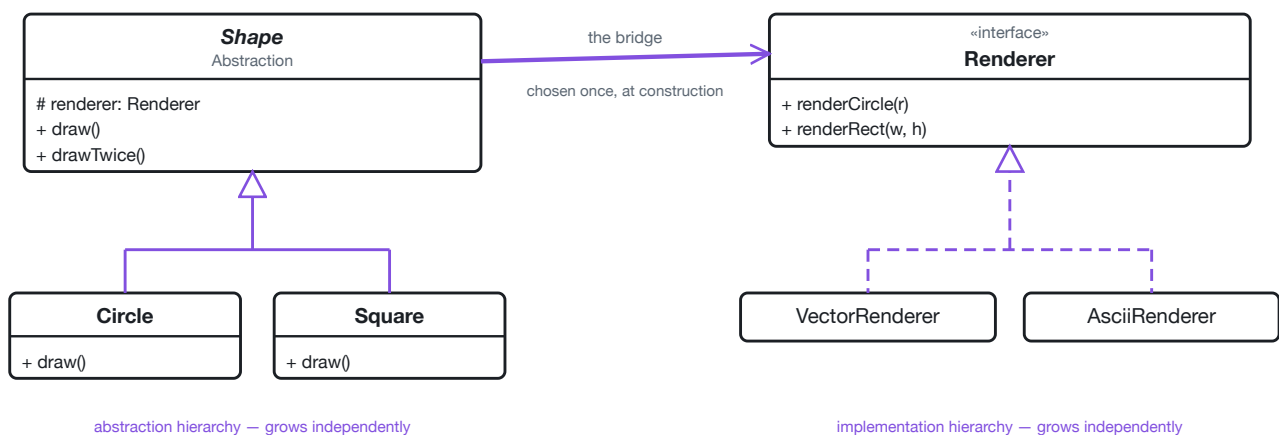
PROBLEM

You have a `Shape` hierarchy with `Circle` and `Square`. Now add colour: `RedCircle`, `BlueCircle`, `RedSquare`, `BlueSquare`. Add `Triangle` and you need two more classes; add `Green` and you need three. The hierarchy grows as **shapes** × **colours**. This is exactly the inheritance blow-up from Part I, and it appears whenever a class varies along two independent axes.

SOLUTION

Pick which axis is the **abstraction** (the one clients talk to and which owns the high-level logic) and which is the **implementation** (the low-level, platform- or mechanism-specific part). Extract the implementation axis into its own interface, and give the abstraction a reference to it. Growth becomes **shapes** + **colours**.

The naming is unfortunate: "implementation" does not mean "subclass". It means a separate object providing primitive operations that the abstraction orchestrates. The reference between them is the "bridge".



Structure. Two hierarchies joined by a single reference; both sides extend without touching the other.

CODE

```
// --- implementation axis: how pixels get made ---
interface Renderer {
  renderCircle(r: number): string
  renderRect(w: number, h: number): string
}
class VectorRenderer implements Renderer {
  renderCircle(r: number) { return `<circle r="${r}" />` }
  renderRect(w: number, h: number) { return `<rect w="${w}" h="${h}" />` }
}
class AsciiRenderer implements Renderer {
  renderCircle(r: number) { return "o".repeat(r) }
  renderRect(w: number, h: number) { return ("#".repeat(w) + "\n").repeat(h) }
}

// --- abstraction axis: what a shape is ---
abstract class Shape {
  constructor(protected renderer: Renderer) {}
  abstract draw(): string
  // High-level logic lives here and works with any renderer.
  drawTwice() { return this.draw() + this.draw() }
}
class Circle extends Shape {
  constructor(r: Renderer, private radius: number) { super(r) }
  draw() { return this.renderer.renderCircle(this.radius) }
}
class Square extends Shape {
  constructor(r: Renderer, private side: number) { super(r) }
  draw() { return this.renderer.renderRect(this.side, this.side) }
}

// Any shape × any renderer, decided at run time.
console.log(new Circle(new VectorRenderer(), 5).draw()) // <circle r="5"/>
console.log(new Circle(new AsciiRenderer(), 5).draw()) // ooooo
```

BRIDGE VS STRATEGY — THEY LOOK IDENTICAL

Structurally they are the same picture: an object holding a swappable collaborator. The difference is intent and granularity. **STRATEGY** swaps *one algorithm* inside an otherwise fixed class, and typically changes per call.

BRIDGE splits an *entire dimension* of a design — the implementor interface is a whole platform-level API, usually chosen once at construction, and both sides are expected to grow into hierarchies. Bridge is an architectural decision; Strategy is a local one.

PROS

- Turns multiplicative class growth into additive.
- Implementation can be swapped at run time.
- Clients see only the abstraction — platform details stay hidden.
- Both hierarchies can be developed by separate teams.

CONS

- Extra indirection makes a cohesive class harder to follow.
- Getting the abstraction/implementor split wrong is expensive to undo.
- Pointless when there is only one implementation and always will be.

In the wild: JDBC (Connection / Statement abstraction over a vendor driver implementation); OS device-driver layers; graphics APIs with swappable rendering backends (OpenGL / Vulkan / Metal behind one scene API). **Relations:** Often introduced together with **Abstract Factory**, which creates matching abstraction/implementor pairs. **Adapter** solves the same shape of problem *after* the fact.

Composite

also: *Object Tree*

INTENT

Compose objects into tree structures and let clients treat individual objects and whole compositions through the same interface.

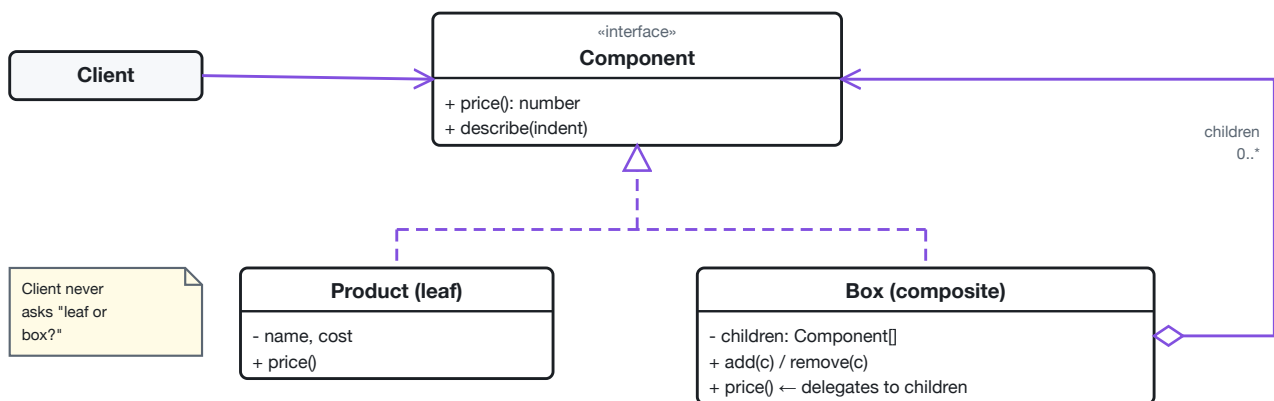
PROBLEM

An order contains products and boxes; boxes contain smaller boxes and products, to arbitrary depth. To compute the total price, client code must unwrap: check the type, branch, recurse manually, keep track of depth. Every operation (price, weight, count, render) repeats that type-checking traversal, and adding a third node type means editing all of them.

SOLUTION

Declare one **component** interface with the operations you need. Implement it twice: a **leaf** that does the real work, and a **composite** that holds children and implements each operation by delegating to them and combining the results.

Now the client calls `component.price()` without knowing or caring whether it is holding one product or a tree ten levels deep. The recursion is inside the structure, not in the client.



Structure. Leaf and composite share one interface; the composite holds children of that same interface, so nesting is unlimited.

CODE

```
interface Component {
  price(): number
  describe(indent?: string): string
}

class Product implements Component {
  constructor(private name: string, private cost: number) {}
  price() { return this.cost }
  describe(i = "") { return `${i}${this.name} - ${this.cost}` }
}

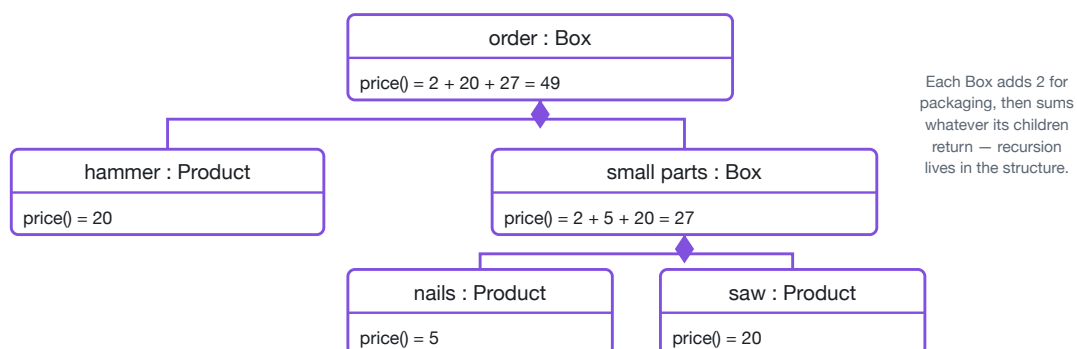
class Box implements Component {
  private children: Component[] = []
  constructor(private label: string, private packaging = 2) {}

  add(...c: Component[]) { this.children.push(...c); return this }
  remove(c: Component) { this.children = this.children.filter(x => x !== c) }

  // Delegate to children, then combine. This is the whole pattern.
  price(): number {
    return this.packaging +
      this.children.reduce((sum, c) => sum + c.price(), 0)
  }
  describe(i = ""): string {
    return [`${i}${this.label} [${this.price()}]`,
      ...this.children.map(c => c.describe(i + " "))].join("\n")
  }
}

const order = new Box("order").add(
  new Product("hammer", 20),
  new Box("small parts").add(
    new Product("nails", 5),
    new Product("saw", 20))
)

// Client does not branch on type and does not recurse.
console.log(order.describe())
console.log(order.price()) // 49
```



Object diagram for the code example: `order.price()` is one call; the tree computes itself.

THE DESIGN TENSION: WHERE DO ADD AND REMOVE GO?

Put them on `Component` and clients get full uniformity — but a leaf must implement `add()` by throwing, which violates Liskov. Put them only on `Composite` (as above) and the types stay honest, but clients must sometimes downcast to build the tree. GoF favours uniformity; most modern code favours type safety, restricting the mutation API to the composite and doing tree-building in one place (often a **BUILDER**).

PROS

- Recursive structures are handled without recursive client code.
- New leaf or composite types need no client changes.
- Operations read as simple one-liners.

CONS

- An over-general component interface loses meaning as node types diverge.
- Hard to constrain which children a composite may hold (type-wise).
- Deep trees can hide expensive traversals behind an innocent-looking call.

In the wild: the DOM; file systems; UI view hierarchies (SwiftUI, Flutter widget trees); AST nodes; arithmetic expression trees; nested menu structures. **Relations:** Frequently traversed with **Iterator** or **Visitor**. Assembled with a **Builder**. **Decorator** looks similar — a wrapper with one child — and is sometimes described as a degenerate composite. **Flyweight** lets you share repeated leaves.

Decorator

also: *Wrapper*

INTENT

Attach new responsibilities to an object dynamically by wrapping it in another object with the same interface — a flexible alternative to subclassing for extending behaviour.

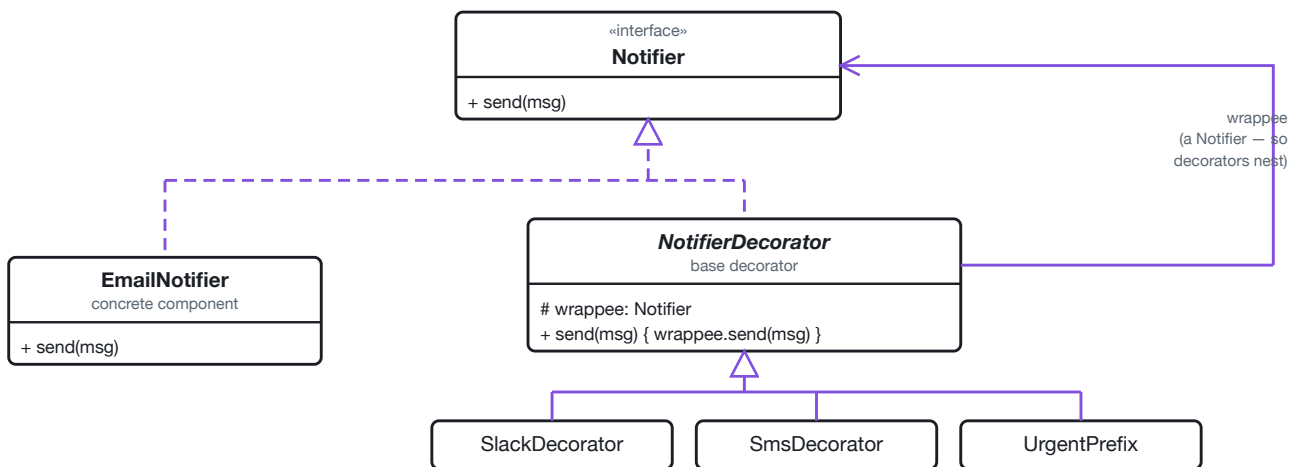
PROBLEM

A notification library sends email. Users now want Slack, SMS, and push — and combinations: "email + Slack", "SMS + push + Slack". Subclassing gives you `EmailAndSlack`, `EmailAndSms`, `EmailAndSlackAndSms` — **2ⁿ classes for n channels**. And a user's chosen combination is known only at run time, which inheritance cannot express at all.

SOLUTION

Define a **decorator** that implements the same interface as the thing it wraps and holds a reference to it. Its method does its own extra work and calls through to the wrapped object. Since a decorator is the interface, it can wrap another decorator — so decorators stack to arbitrary depth, in any order, chosen at run time.

The linked list of wrappers ends at a real "concrete component" that does the base work.



Structure. The base decorator both implements and holds the component interface — that is what makes stacking possible.

CODE

```
interface Notifier { send(msg: string): void }

// The base component: does the real, minimal work.
class EmailNotifier implements Notifier {
  send(msg: string) { console.log("email: " + msg) }
}

// The base decorator: implements Notifier by holding a Notifier.
abstract class NotifierDecorator implements Notifier {
  constructor(protected wrappee: Notifier) {}
  send(msg: string) { this.wrappee.send(msg) }
}

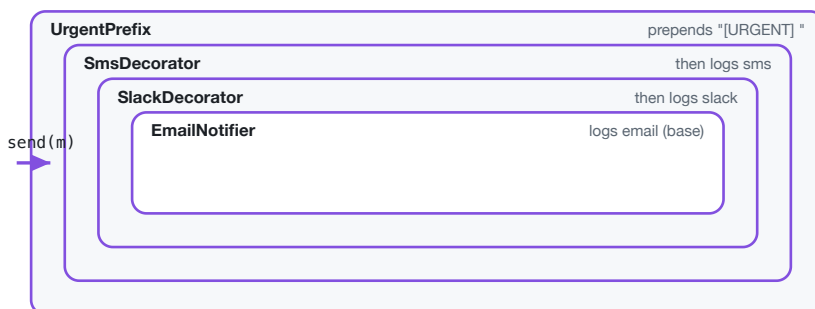
class SlackDecorator extends NotifierDecorator {
  send(msg: string) {
    super.send(msg)           // delegate first...
    console.log("slack: " + msg) // ...then add behaviour
  }
}

class SmsDecorator extends NotifierDecorator {
  send(msg: string) { super.send(msg); console.log("sms: " + msg) }
}

// Decorators need not just append – they can transform or short-circuit.
class UrgentPrefix extends NotifierDecorator {
  send(msg: string) { super.send("[URGENT] " + msg) }
}

// Composed from user preferences at run time.
function build(channels: string[]): Notifier {
  let n: Notifier = new EmailNotifier()
  if (channels.includes("slack")) n = new SlackDecorator(n)
  if (channels.includes("sms"))  n = new SmsDecorator(n)
  return new UrgentPrefix(n)
}

build(["slack", "sms"]).send("disk almost full")
// email: [URGENT] disk almost full
// slack: [URGENT] disk almost full
// sms:   [URGENT] disk almost full
```



Call order

- 1 Urgent.send("disk full")
- 2 → Sms.send("[URGENT] disk full")
- 3 → Slack.send(...)
- 4 → Email.send(...) logs email
- 5 ← Slack logs slack
- 6 ← Sms logs sms

Each wrapper delegates inward, then adds its own work.

Runtime. The object built by `build(["slack", "sms"])` is an onion; the client holds only the outermost layer.

PROS

- Extend behaviour without subclassing or editing the original class.
- Add and remove responsibilities at run time.
- Combine many behaviours by stacking; n features, not 2^n classes.
- Each decorator is small and single-purpose (SRP).

CONS

- Removing a specific wrapper from the middle of a stack is awkward.
- Behaviour depends on **order**, which is easy to get wrong and hard to see.
- Debugging a deep stack means stepping through many near-identical frames.
- Wrappers are not `===` the wrapped object — identity checks break.

In the wild: Java I/O streams — `new BufferedInputStream(new GZIPInputStream(new FileInputStream(f)))` is two decorators over a concrete component (note that `InputStreamReader`, which turns bytes into chars, is really an **Adapter**: it changes the interface); HTTP middleware chains in Express, Koa, ASP.NET; Python function decorators; React higher-order components. **Relations:** Same structure as **Proxy**, different purpose (add behaviour vs. control access) and lifecycle (client composes decorators; a proxy usually manages its subject itself). **Chain of Responsibility** also builds a chain, but any link may stop it; decorators all run. **Adapter** changes the interface; Decorator preserves it.

Facade

INTENT

Provide a single, simplified interface to a complex subsystem of many classes.

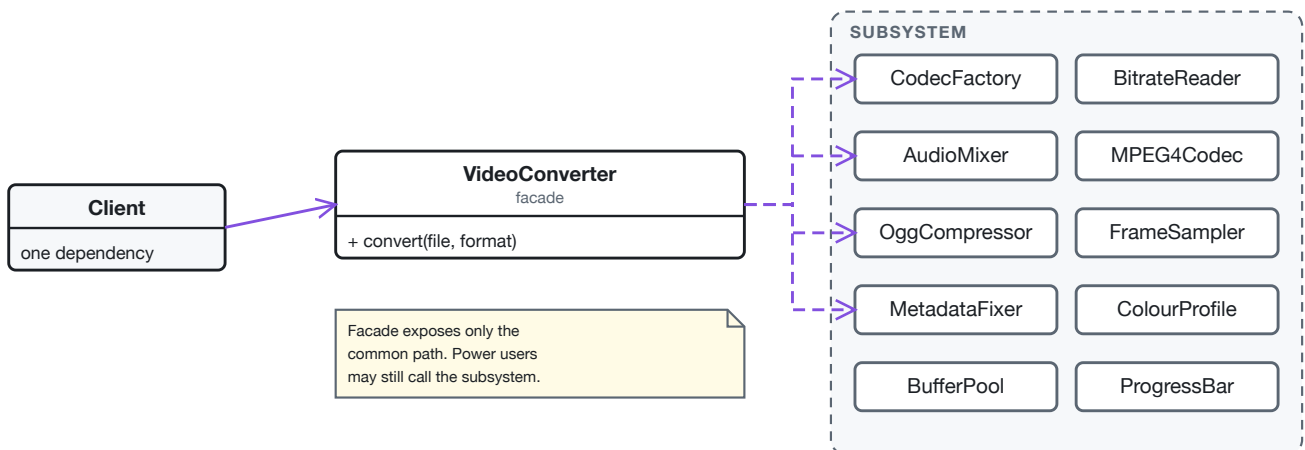
PROBLEM

To convert a video you must instantiate a codec factory, sniff the format, build a bitrate reader, allocate an audio mixer, run the conversion, then fix the result file's metadata — a dozen objects, in a strict order, with interdependencies. Any code needing "convert this video" must reproduce all that, becoming coupled to a dozen classes it doesn't care about.

SOLUTION

Write one class whose methods express what callers actually want (`convert(file, "mp4")`) and whose bodies contain the orchestration. The facade owns the subsystem's initialisation order and lifecycle. Clients depend on one small class instead of twelve large ones.

A facade deliberately exposes only a useful subset. If a caller needs the full power of the subsystem, they can still reach past the facade — it restricts nothing, it just gives the common path a name.



Structure. The client depends on one class; the facade depends on many. The subsystem does not know the facade exists.

CODE

```
// --- the messy subsystem (abridged) ---
class CodecFactory { static extract(f: string) { return f.endsWith(".mp4") ? "mpeg4" : "ogg" } }
class BitrateReader { static read(f: string, c: string) { return `raw(${f},${c})` }
    static convert(b: string, c: string) { return `conv(${b}→${c})` } }
class AudioMixer { fix(s: string) { return `mixed(${s})` } }
class MetadataFixer { stamp(s: string) { return `${s}+meta` } }

// --- the facade ---
class VideoConverter {
    convert(filename: string, target: string): string {
        const sourceCodec = CodecFactory.extract(filename)
        const buffer = BitrateReader.read(filename, sourceCodec)
        let result = BitrateReader.convert(buffer, target)
        result = new AudioMixer().fix(result)
        return new MetadataFixer().stamp(result)
    }
}

// --- client: one line, one dependency ---
console.log(new VideoConverter().convert("holiday.ogg", "mp4"))
```

THE GOD-OBJECT RISK

A facade is the pattern most likely to rot. It starts as five methods and becomes a 3,000-line class that every module depends on — the coupling you were avoiding, now centralised. Countermeasures: keep facades **TASK-SCOPED** (`VideoConverter`, not `MediaManager`), allow several small facades over one subsystem, and never let business logic accumulate in a facade — it orchestrates, it does not decide.

PROS

- Isolates clients from subsystem complexity and churn.
- Reduces compile/deploy coupling to a single class.
- Gives the common workflow a discoverable name.
- Makes the subsystem replaceable behind a stable seam.

CONS

- Tends toward a god object.
- Can hide capability that advanced callers need.
- Adds a layer with no behaviour of its own to test.

In the wild: `fetch()` over the HTTP stack; `jQuery` over the DOM; `Files.readString()` over Java NIO; SDK client classes wrapping raw REST calls; an application-service layer over a domain model. **Relations:** Frequently a **Singleton** (one facade instance suffices). **Adapter** wraps one object to change its interface; Facade wraps many to simplify. **Abstract Factory** can hide subsystem creation, which is a facade over construction specifically. Compare **Mediator**: a mediator's colleagues know only the mediator, whereas subsystem classes are unaware the facade exists.

Flyweight

also: *Cache*

INTENT

Fit more objects into available memory by sharing the parts of state that are common between them, instead of storing that state in every object.

PROBLEM

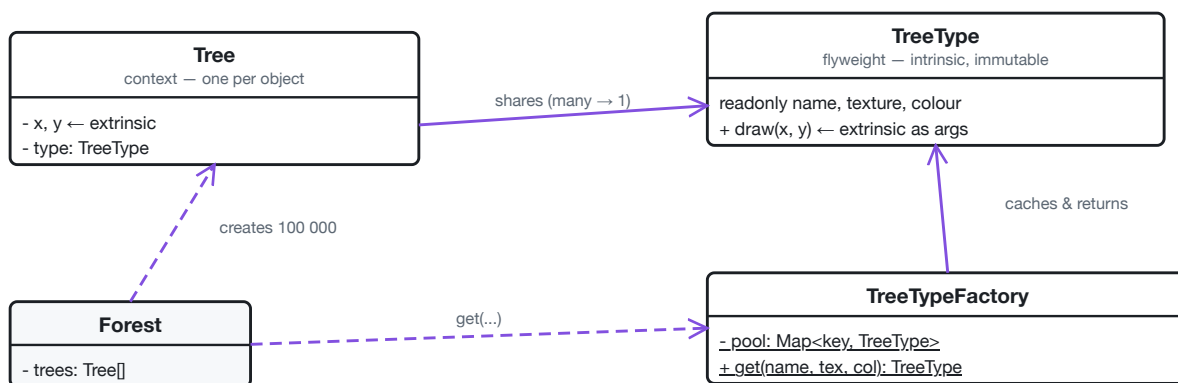
A game renders a forest of 1,000,000 trees. Each `Tree` holds coordinates, plus a name, a texture bitmap, and a colour. The bitmap is a megabyte. There are three species. You are therefore storing the same three bitmaps a million times — roughly a terabyte of duplicated data, when three megabytes would do.

SOLUTION

Split each object's fields into two groups:

- **Intrinsic state** — identical across many objects and never changes (species name, texture, colour). Extract it into a separate, **immutable flyweight** object.
- **Extrinsic state** — unique per object (x, y, height). Keep it in the many lightweight objects, or pass it into flyweight methods as arguments.

A **flyweight factory** holds a pool keyed by intrinsic state and returns an existing flyweight whenever one matches, creating a new one only for a genuinely new combination. Immutability is mandatory: shared objects that can be mutated would corrupt every holder at once.



Structure. Contexts hold unique data and a pointer to a shared, immutable flyweight; the factory guarantees one flyweight per intrinsic-state key.

CODE

```
// Flyweight: intrinsic, shared, IMMUTABLE.
class TreeType {
  constructor(
    readonly name: string,
    readonly texture: string,    // the expensive megabyte
    readonly colour: string) {}

  // Extrinsic state arrives as parameters, never stored.
  draw(x: number, y: number) {
    return `${this.name}@${x},${y} [${this.texture}]`
  }
}

// Flyweight factory: the only way to obtain a TreeType.
class TreeTypeFactory {
  private static pool = new Map<string, TreeType>()

  static get(name: string, texture: string, colour: string): TreeType {
    const key = `${name}|${texture}|${colour}`
    let t = TreeTypeFactory.pool.get(key)
    if (!t) {
      t = new TreeType(name, texture, colour)
      TreeTypeFactory.pool.set(key, t)
    }
    return t // shared instance
  }
  static size() { return TreeTypeFactory.pool.size }
}

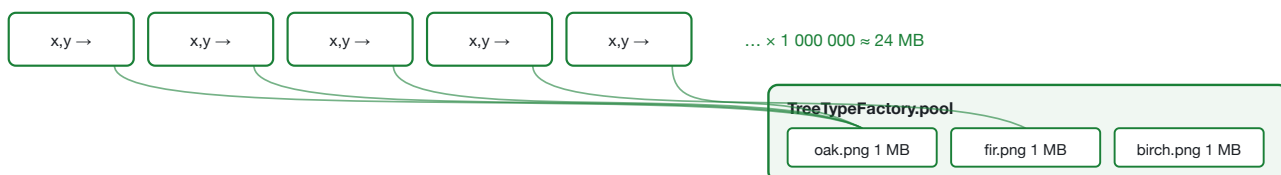
// The many lightweight context objects.
class Tree {
  constructor(private x: number, private y: number, private type: TreeType) {}
  draw() { return this.type.draw(this.x, this.y) }
}

const forest: Tree[] = []
for (let i = 0; i < 100_000; i++) {
  const species = i % 3 === 0 ? "oak" : i % 3 === 1 ? "fir" : "birch"
  forest.push(new Tree(i % 800, i % 600,
    TreeTypeFactory.get(species, species + ".png", "green")))
}
console.log(forest.length, TreeTypeFactory.size()) // 100000 3
```

BEFORE — each tree stores its own 1 MB texture



AFTER — trees hold coordinates + a pointer; 3 textures exist in total



Memory picture. Intrinsic state moves into three shared objects; extrinsic state stays per tree.

APPLY THIS LAST, AND ONLY WITH A MEASUREMENT

Flyweight trades CPU (pool lookups, passing extrinsic state around) and a large amount of clarity for RAM. It is worth it only when you have measured that duplicated intrinsic state is genuinely the memory problem — which usually means tens of thousands of objects or more. Introducing it speculatively produces code that is harder to read for no benefit. Also: an unbounded pool is a memory leak. Bound it, or use weak references.

PROS

- Can cut memory use by orders of magnitude in the right situation.
- Fewer allocations means less GC pressure.
- Immutability makes flyweights inherently thread-safe.

CONS

- Trades RAM for CPU and complexity.
- Code becomes harder to follow: state is split across two objects.
- Flyweights must be immutable — enforced only by discipline.
- An unbounded pool never releases memory.

In the wild: Java's `String` interning and `Integer.valueOf` cache ($-128\dots127$); Python's small-int and interned-string caches; glyph/font caches in text renderers; tile and sprite atlases in games; immutable value objects in a DDD codebase.

Relations: The factory is usually a **Singleton**. **Composite** trees often share repeated leaves as flyweights. Resembles **Singleton** but holds many shared instances rather than one.

Proxy

INTENT

Provide a stand-in for another object that controls access to it, letting you do something before or after the request reaches the real object.

PROBLEM

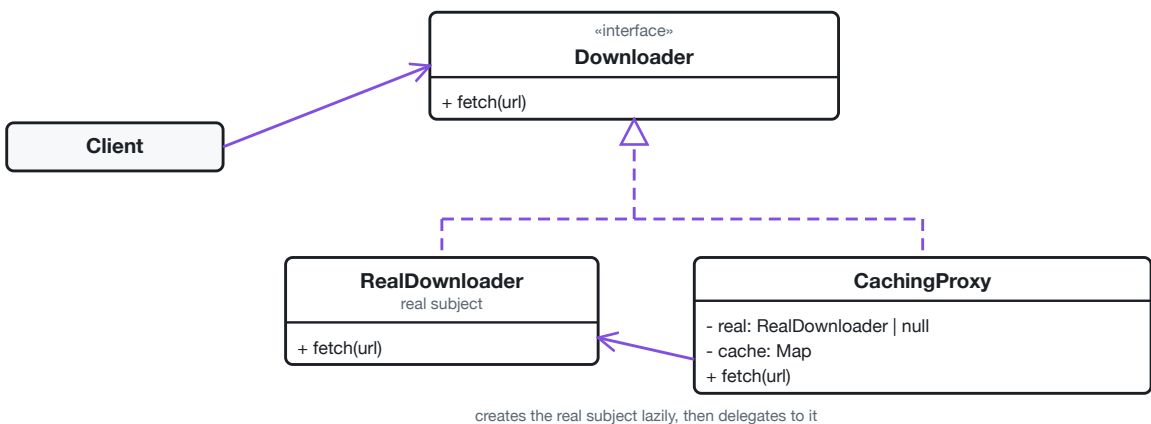
A `VideoDownloader` object connects to a remote service on construction and is slow to create. Many clients need it, but most sessions use only a fraction of them. You want lazy creation — and separately, you want caching, access checks, and request logging. You could add all of that to the downloader itself, but then it violates SRP, and you cannot change the policy without editing a class you may not own.

SOLUTION

Create a class with the *same interface* as the real service. It holds a reference to the real object (often creating it on first use) and forwards calls, doing its extra work around the delegation. Clients cannot tell the difference, so a proxy can be dropped in with zero changes.

THE STANDARD PROXY VARIETIES

KIND	WHAT IT DOES
Virtual (lazy)	Defers creating the expensive real object until a method is actually called.
Remote	Represents an object in another process or machine; marshals the call over the network. This is what an RPC stub is.
Protection	Checks permissions before forwarding.
Caching	Stores results and answers repeat requests without touching the real object.
Logging / metering	Records calls, timings, and counts.
Smart reference	Tracks live clients so the real object can be released when unused.



Structure. Same interface as the subject, so the client cannot tell the difference — and need not change.

CODE

```
interface Downloader { fetch(url: string): string }

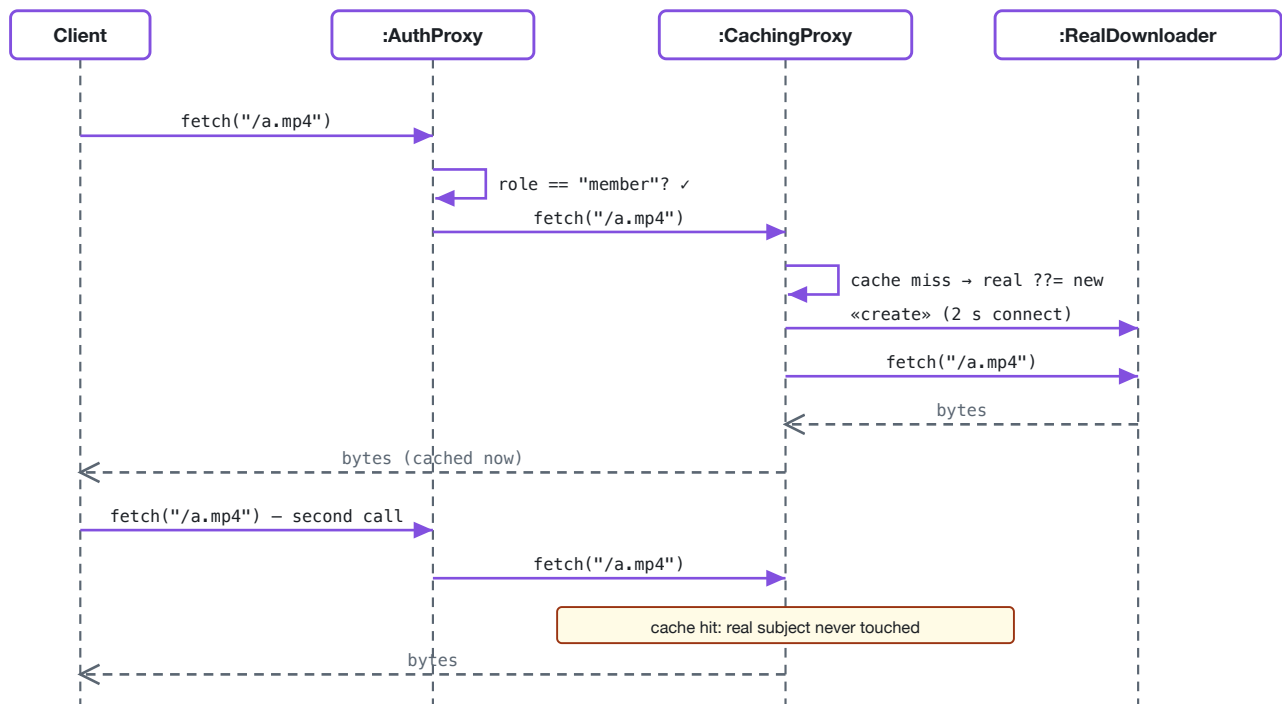
// The real subject: expensive to build, expensive to call.
class RealDownloader implements Downloader {
  constructor() { console.log(" [connecting... 2s]") }
  fetch(url: string) { console.log(" [network hit]"); return `bytes(${url})` }
}

// Virtual + caching + logging proxy, all in one.
class CachingProxy implements Downloader {
  private real: RealDownloader | null = null // not built yet
  private cache = new Map<string, string>()

  fetch(url: string): string {
    if (this.cache.has(url)) {
      console.log(" [cache hit]")
      return this.cache.get(url)! // real object never touched
    }
    this.real ??= new RealDownloader() // lazy init
    const data = this.real.fetch(url)
    this.cache.set(url, data)
    return data
  }
}

// A protection proxy composes with it – same interface, so it stacks.
class AuthProxy implements Downloader {
  constructor(private next: Downloader, private role: string) {}
  fetch(url: string) {
    if (this.role !== "member") throw new Error("forbidden")
    return this.next.fetch(url)
  }
}

const d: Downloader = new AuthProxy(new CachingProxy(), "member")
d.fetch("/a.mp4") // [connecting... 2s] [network hit]
d.fetch("/a.mp4") // [cache hit]
```



Runtime. Two stacked proxies: the second call never reaches the real object — the signature of a proxy rather than a decorator.

PROXY VS DECORATOR

Identical structure; the distinction is in *purpose* and *control*. A **DECORATOR** adds behaviour and is composed by *the client*, which supplies the wrapped object. A **PROXY** controls access and typically *manages the subject's lifecycle itself* — the client may not even know the real object exists. Practical test: if the wrapper can decide not to call the real object at all (cache hit, denied permission), it's a proxy.

PROS

- Control the subject transparently — clients need no changes.
- Manage the subject's lifecycle independently of clients.
- Keeps cross-cutting concerns (auth, cache, logging) out of business classes.
- Works even when the subject isn't ready or isn't local.

CONS

- More classes and indirection.
- Lazy initialisation can move latency somewhere surprising.
- Cache invalidation becomes your problem.
- Stack traces get deeper and less readable.

In the wild: ORM lazy-loading proxies (Hibernate, Entity Framework); JavaScript's `Proxy` object and Vue's reactivity system; gRPC/Thrift client stubs; Spring AOP proxies implementing `@Transactional` and `@Cacheable`; service meshes (a network-level protection + logging proxy). **Relations:** Same structure as **Decorator** and **Adapter**; differs in purpose. **Facade** also shields clients, but does not implement the subsystem's interface and cannot be substituted for it.

Part IV — Behavioral Patterns

Behavioral patterns are about **responsibility and communication**: who does what, and how objects talk without becoming welded together. This is the largest category and the one that pays off most in day-to-day code, because most maintenance pain is really communication pain — a change here forces a change there.

Three recurring moves show up across the eleven:

MOVE	PATTERNS	EFFECT
Extract a varying decision into an object	Strategy, State, Command, Interpreter	Replaces conditionals with polymorphism
Insert an indirection between senders and receivers	Observer, Mediator, Chain of Responsibility	Removes direct references between collaborators
Move an operation out of the object it acts on	Visitor, Iterator, Memento, Template Method	Lets data and algorithms evolve separately

Chain of Responsibility

also: CoR, Chain of Command

INTENT

Pass a request along a chain of handlers; each decides whether to process it, pass it on, or stop the chain — so the sender never knows which handler will deal with it.

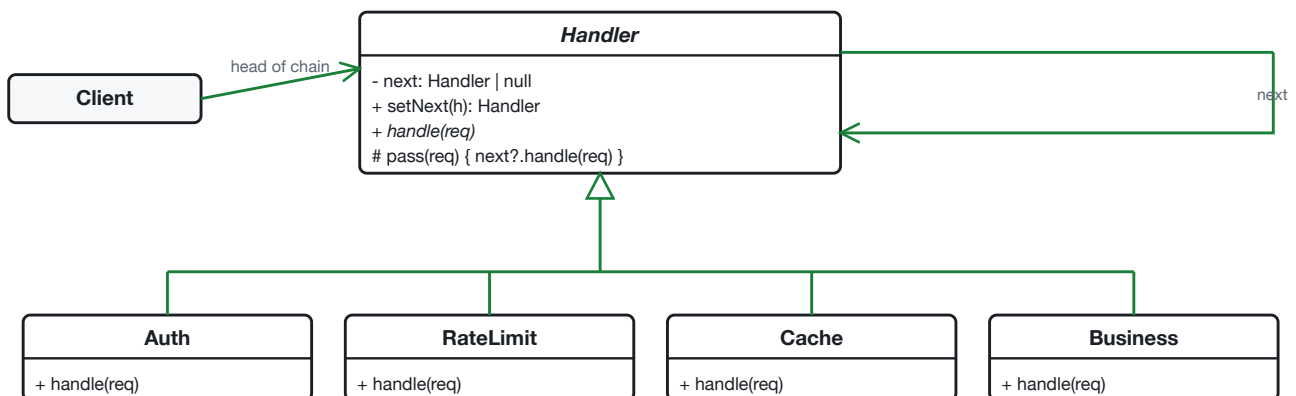
PROBLEM

An order endpoint must, before touching business logic: authenticate, check permissions, validate the payload, apply rate limiting, and serve from cache. Written inline, that is a 200-line method of nested `if`s. Every new check makes it worse, the order of checks is implicit, and none of the checks can be reused by another endpoint or tested alone.

SOLUTION

Turn each check into a standalone **handler** object with one method. Each handler holds a reference to the *next* handler. Its method does its own work and then either calls `next.handle(request)` or returns without doing so, ending the chain.

The chain is assembled at run time from configuration, so order is explicit and rearranging it is a data change, not a code change. A handler that cannot cope simply passes the request on — no handler needs to know the full pipeline.



Concrete handlers either finish the request or call `pass(req)`.

Structure. A self-referencing linked list of handlers behind one interface; the client sees only the head.

CODE

```
type Req = { user?: string; path: string; hits: number }

abstract class Handler {
  private next: Handler | null = null

  setNext(h: Handler): Handler { this.next = h; return h } // chainable

  abstract handle(r: Req): string

  protected pass(r: Req): string {
    return this.next ? this.next.handle(r) : "404 no handler"
  }
}

class Auth extends Handler {
  handle(r: Req) {
    if (!r.user) return "401 unauthenticated" // stop the chain
    return this.pass(r)
  }
}

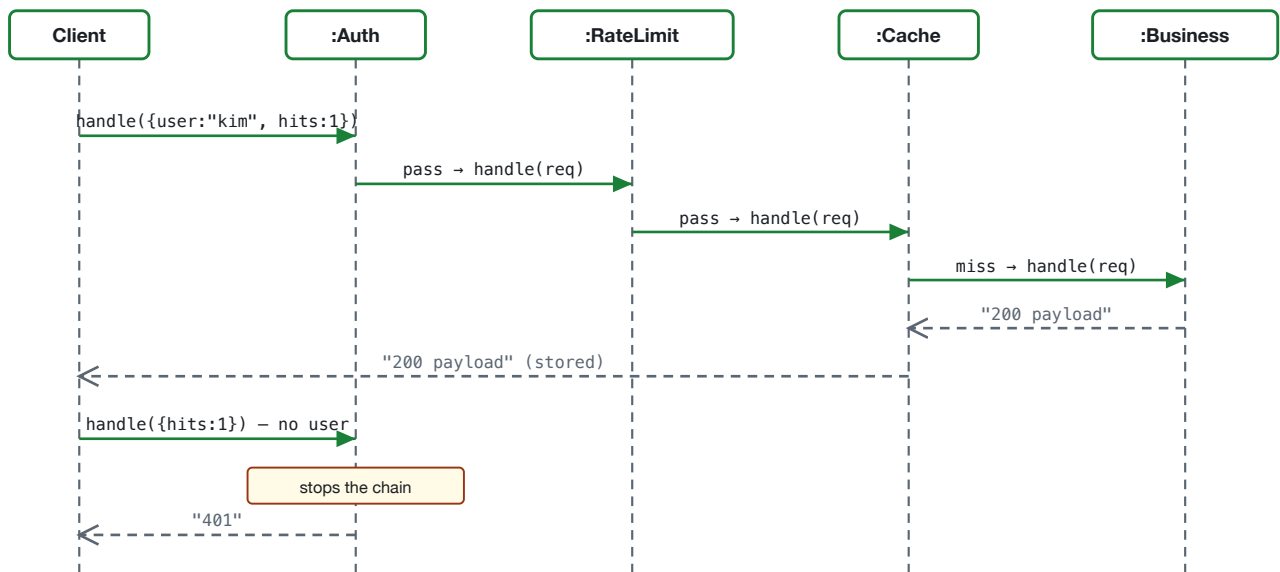
class RateLimit extends Handler {
  constructor(private max: number) { super() }
  handle(r: Req) {
    if (r.hits > this.max) return "429 too many requests"
    return this.pass(r)
  }
}

class Cache extends Handler {
  private store = new Map<string, string>()
  handle(r: Req) {
    const hit = this.store.get(r.path)
    if (hit) return "200 (cached) " + hit
    const fresh = this.pass(r)
    this.store.set(r.path, fresh)
    return fresh // work after delegating, too
  }
}

class Business extends Handler {
  handle(r: Req) { return `200 payload for ${r.path}` }
}

// Assemble – order is explicit and reconfigurable.
const auth = new Auth()
auth.setNext(new RateLimit(100)).setNext(new Cache()).setNext(new Business())

console.log(auth.handle({ path: "/orders", hits: 1 })) // 401
console.log(auth.handle({ user: "kim", path: "/orders", hits: 1 })) // 200 payload
console.log(auth.handle({ user: "kim", path: "/orders", hits: 2 })) // 200 (cached)
```



Runtime. The first request walks the whole chain; the second is rejected by the first link and never reaches the rest.

TWO FLAVOURS

PURE COR (GoF): exactly one handler processes the request, then the chain stops — think exception handling or event bubbling.

PIPELINE COR (dominant in practice): every handler contributes something and passes along, with early exit as an option. This is what HTTP middleware is. Both are called Chain of Responsibility.

PROS

- Decouples sender from receiver entirely.
- Order and membership are run-time data.
- Each handler is small, reusable, and independently testable (SRP).
- New handlers need no changes to existing ones (OCP).

CONS

- A request can fall off the end unhandled — silently, if you're careless.
- Control flow is hard to follow; no single place shows the whole pipeline.
- Debugging means stepping through many hops.
- Performance cost if chains are long and hot.

In the wild: Express/Koa/ASP.NET middleware; Java Servlet filters; DOM event bubbling; exception propagation up the call stack; multi-level approval workflows. **Relations:** Often built on a **Composite** (a parent is the natural "next"). Structurally close to **Decorator**, but a decorator always delegates whereas a CoR link may not. Handlers are frequently implemented with **Command**.

Command

also: Action, Transaction

INTENT

Turn a request into a stand-alone object containing everything needed to perform it — enabling queues, logs, undo, and parameterising objects with operations.

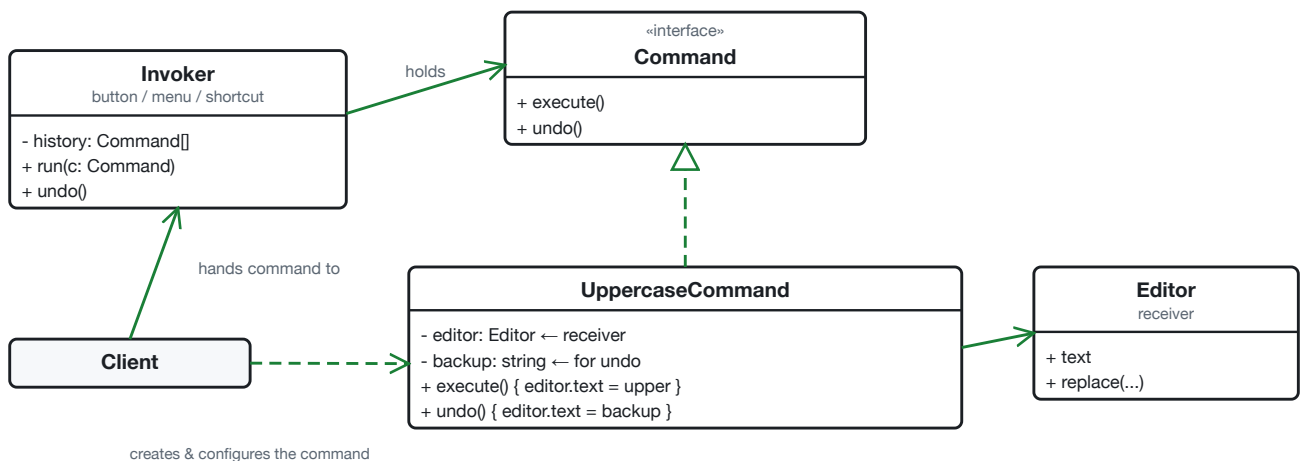
PROBLEM

A text editor's toolbar button, context menu item, and keyboard shortcut all "copy". Subclassing `Button` for each action gives you dozens of near-identical classes, and duplicating the copy logic in three places guarantees they drift. Worse, the UI layer ends up holding direct references to the business layer, so every UI element knows how the editor works internally.

SOLUTION

Extract the request into a **command object** with one method, `execute()`. The command holds its receiver and any parameters it needs — it is a self-contained, already-bound function call. Now the button holds a `Command` and knows nothing else; the same command instance serves the menu and the shortcut.

Because the call is now an *object*, you can do things you cannot do with a method call: put it in a queue, send it over a network, write it to a log, keep a history of it, and — by adding an `undo()` method that reverses the effect — get undo/redo almost for free.



Structure. The invoker knows only the `Command` interface; the command knows its receiver; UI and business logic never reference each other.

CODE

```
// Receiver: the real business logic. Knows nothing about commands.
class Editor {
    text = "hello world"
    replace(from: number, to: number, with_: string) {
        this.text = this.text.slice(0, from) + with_ + this.text.slice(to)
    }
}

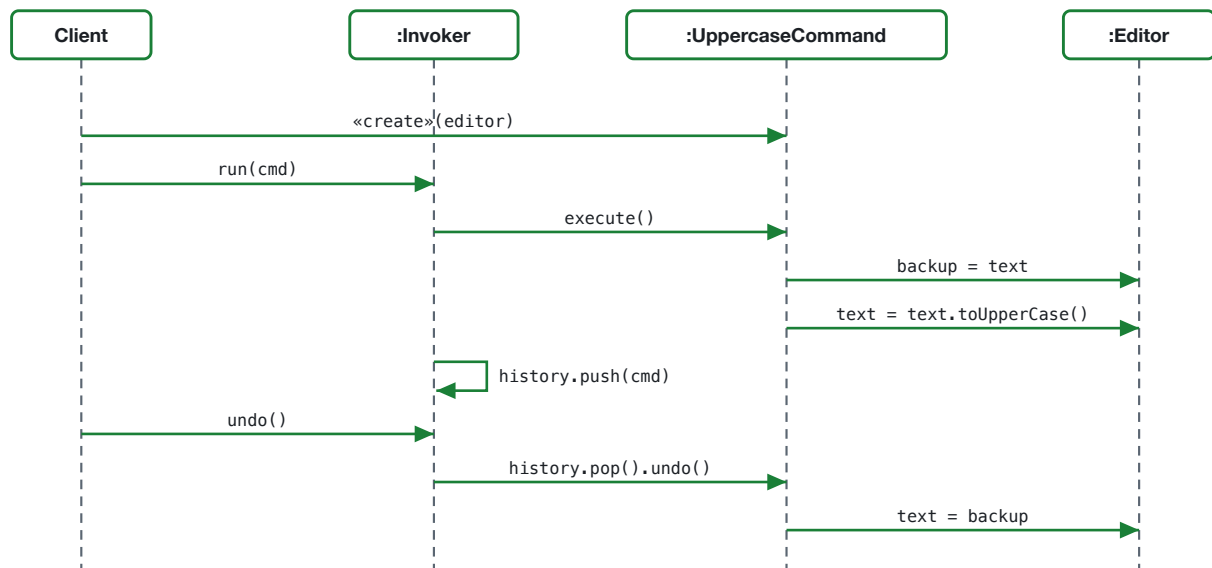
interface Command {
    execute(): void
    undo(): void
}

class UppercaseCommand implements Command {
    private backup = "" // state needed to reverse
    constructor(private editor: Editor) {}
    execute() {
        this.backup = this.editor.text
        this.editor.text = this.editor.text.toUpperCase()
    }
    undo() { this.editor.text = this.backup }
}

class InsertCommand implements Command {
    private backup = ""
    constructor(private editor: Editor, private at: number, private s: string) {}
    execute() { this.backup = this.editor.text; this.editor.replace(this.at, this.at, this.s) }
    undo() { this.editor.text = this.backup }
}

// Invoker: triggers commands and owns the history. Knows no receiver.
class Invoker {
    private history: Command[] = []
    run(c: Command) { c.execute(); this.history.push(c) }
    undo() { this.history.pop()?.undo() }
}

const ed = new Editor(), inv = new Invoker()
inv.run(new InsertCommand(ed, 5, ","))
inv.run(new UppercaseCommand(ed))
console.log(ed.text) // HELLO, WORLD
inv.undo(); inv.undo()
console.log(ed.text) // hello world
```



Runtime. Because the request is an object, the invoker can keep it and reverse it later.

UNDO: SNAPSHOT OR INVERSE?

Storing a full backup (above) is simple and always correct, but memory-heavy for large documents — that is **MEMENTO** territory. Storing an *inverse operation* ("delete the 1 char at position 5") is compact but must be exactly right for every command, including edge cases. Editors typically use inverses; databases use write-ahead logs of both.

PROS

- Decouples the thing that invokes from the thing that performs.
- Commands are first-class: queue, log, serialise, schedule, retry them.
- Undo/redo becomes structural rather than ad hoc.
- Compose simple commands into macro commands (via **Composite**).

CONS

- A class per operation — a lot of ceremony for trivial actions.
- Correct undo is genuinely hard once commands interact.
- In languages with closures, a plain function often does the job.

In the wild: every editor's undo stack; job queues (Sidekiq, Celery, Hangfire); Redux actions; database transaction logs; CQRS command handlers; Runnable submitted to an ExecutorService. **Relations:** **Memento** stores the state a command needs to undo. **Composite** makes macro commands. **Chain of Responsibility** handlers are often commands. **Strategy** looks the same but answers "how do I do this", where Command answers "what should be done, and when".

Iterator

INTENT

Provide a way to traverse the elements of a collection sequentially without exposing its underlying representation.

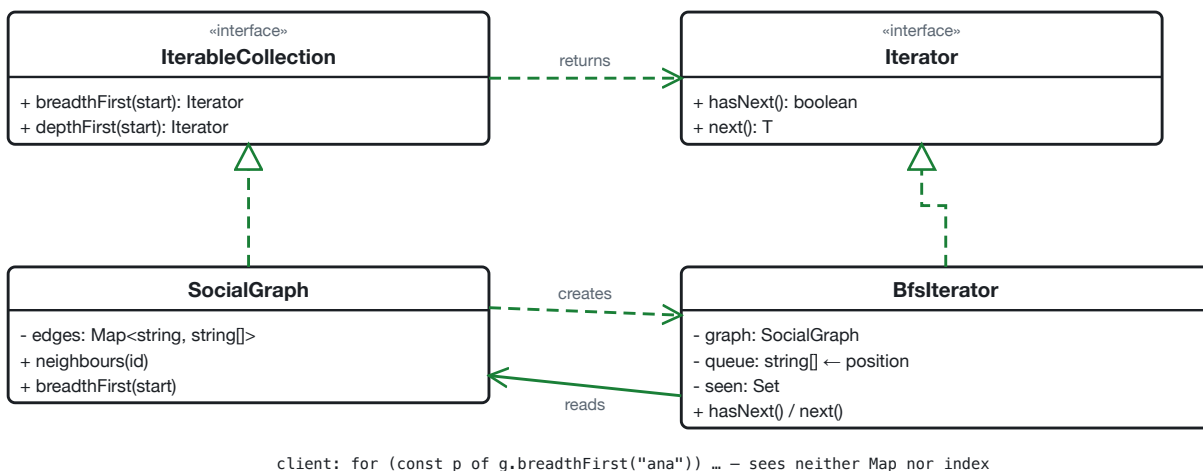
PROBLEM

An array, a hash map, a linked list, and a tree all "contain elements", but traversing each requires different code. If client code loops using indices, it is welded to the array representation. And a tree has several equally valid traversals — depth-first, breadth-first, in order — none of which is naturally "the" way to loop.

SOLUTION

Extract the traversal into its own **iterator** object holding the current position and the rules for advancing. The collection exposes a factory method returning an iterator; the client asks only "is there more?" and "give me the next one".

One collection can now offer several iterators, and each keeps its own position, so multiple traversals can run over the same collection at once.



Structure. Two parallel interfaces: collections produce iterators (a factory method), iterators hold traversal position and read the collection.

CODE

```
// A collection whose internals nobody should depend on.
class SocialGraph {
  private edges = new Map<string, string[]>()
  connect(a: string, b: string) {
    this.edges.set(a, [...(this.edges.get(a) ?? []), b])
  }
  neighbours(id: string) { return this.edges.get(id) ?? [] }

  // Two traversals, same data – the point of the pattern.
  breadthFirst(start: string) { return new BfsIterator(this, start) }
  depthFirst(start: string) { return new DfsIterator(this, start) }
}

class BfsIterator {
  private queue: string[]
  private seen = new Set<string>()
  constructor(private g: SocialGraph, start: string) { this.queue = [start] }

  hasNext() { return this.queue.length > 0 }
  next(): string {
    const id = this.queue.shift()!
    this.seen.add(id)
    for (const n of this.g.neighbours(id))
      if (!this.seen.has(n)) this.queue.push(n)
    return id
  }

  // Bonus: make it work with `for...of` by implementing the JS protocol.
  [Symbol.iterator]() {
    return { next: () => this.hasNext()
      ? { value: this.next(), done: false }
      : { value: undefined, done: true } }
  }
}

class DfsIterator { /* identical but uses pop() instead of shift() */ }

const g = new SocialGraph()
g.connect("ana", "bo"); g.connect("ana", "cy"); g.connect("bo", "dev")

for (const person of g.breadthFirst("ana")) console.log(person)
// ana bo cy dev – client never saw a Map or an array index
```

YOUR LANGUAGE ALREADY HAS THIS

Iterator is the most thoroughly absorbed pattern in the catalogue: `for...of` + `Symbol.iterator` (JS), `__iter__` (Python), `IEnumerable` (C#), `Iterable` (Java), `Iterator` trait (Rust). **GENERATORS** (`function*`, `yield`) collapse a whole iterator class into a few lines and give you laziness for free. Write a hand-rolled iterator class only when you need traversal state the language's protocol can't express. Knowing the pattern still matters — it explains *why* those constructs exist.

PROS

- Client code works with any collection through one interface.
- Traversal algorithms live apart from storage (SRP).
- Multiple independent traversals in parallel.
- Lazy iteration allows infinite or streamed sequences.

CONS

- Overkill for a plain array.
- Direct index access can be faster in hot loops.
- Mutating a collection mid-iteration invalidates iterators — a classic bug (`ConcurrentModificationException`).

Relations: **Composite** trees are typically traversed with an iterator. **Factory Method** is how a collection produces the right iterator type. **Visitor** and **Iterator** are often combined: the iterator walks, the visitor acts. **Memento** can capture an iterator's position.

Mediator

also: Controller, Intermediary

INTENT

Reduce chaotic dependencies between objects by forcing them to collaborate only through a mediator object, instead of referring to each other directly.

PROBLEM

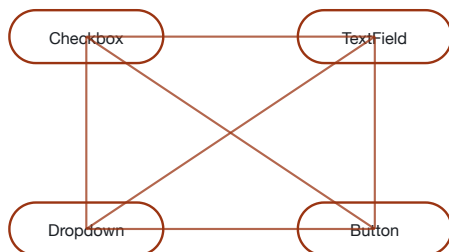
A dialog has a checkbox, a text field, a dropdown, and a submit button. Ticking the checkbox enables the text field. Choosing a dropdown option hides the checkbox. The submit button enables only when the field is non-empty. Implemented naively, each widget holds references to several others. With n widgets you approach n^2 relationships, none of the widgets is reusable elsewhere, and adding one rule means touching several classes.

SOLUTION

Cut all direct links between the components. Each component gains one reference — to the **mediator** — and notifies it when something happens: `mediator.notify(this, "click")`. The mediator holds the interaction rules and calls back into whichever components need to change.

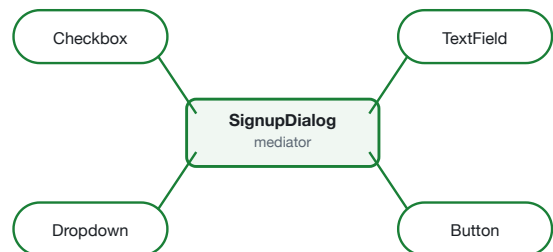
Components become independent and reusable; all the coordination logic sits in one readable place.

BEFORE — every component references its peers



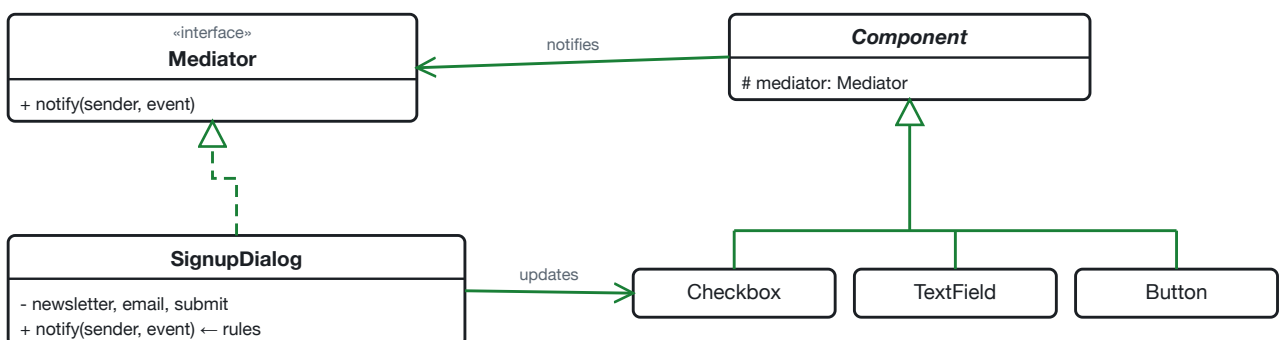
n components $\Rightarrow n(n-1)/2$ links

AFTER — components know only the mediator



n components $\Rightarrow n$ links; rules live in one place

Coupling before and after. The mesh becomes a hub.



Structure. Components hold one Mediator reference and report events; the concrete mediator holds the components and the interaction rules.

CODE

```
interface Mediator { notify(sender: Component, event: string): void }

abstract class Component {
  constructor(protected m: Mediator) {}
  enabled = true
}

class Checkbox extends Component {
  checked = false
  toggle() { this.checked = !this.checked; this.m.notify(this, "toggle") }
}

class TextField extends Component {
  value = ""
  type(s: string) { this.value = s; this.m.notify(this, "input") }
}

class Button extends Component {
  click() { if (this.enabled) this.m.notify(this, "click") }
}

// All interaction rules live here, and only here.
class SignupDialog implements Mediator {
  newsletter = new Checkbox(this)
  email      = new TextField(this)
  submit     = new Button(this)

  constructor() { this.email.enabled = false; this.submit.enabled = false }

  notify(sender: Component, event: string) {
    if (sender === this.newsletter && event === "toggle") {
      this.email.enabled = this.newsletter.checked
      this.submit.enabled = false
    }
    if (sender === this.email && event === "input") {
      this.submit.enabled = this.email.value.includes("@")
    }
    if (sender === this.submit && event === "click") {
      console.log("subscribing " + this.email.value)
    }
  }
}

const d = new SignupDialog()
d.newsletter.toggle(); d.email.type("ana@example.com"); d.submit.click()
```

THE MEDIATOR BECOMES A GOD OBJECT

This is the pattern's occupational hazard: you removed n^2 coupling by concentrating all of it in one class, which then grows without bound. Mitigations: one mediator per cohesive interaction cluster rather than one per screen; move genuine domain rules out into domain objects and keep only coordination in the mediator; and if the mediator's `notify` becomes a giant dispatch switch, that is the signal to split it.

PROS

- Components become independent and reusable.
- Interaction rules are in one place, easy to read and change.
- Cuts coupling from n^2 to n .

CONS

- The mediator can turn into a god object.
- Indirection obscures what actually happens on a click.
- Overkill for two or three components.

In the wild: dialog controllers in GUI toolkits (Swing, JavaFX, WinForms); an air-traffic control tower coordinating aircraft that never talk to each other; chat-room servers; MVP presenters. **Relations:** **Observer** is often the delivery mechanism inside a mediator — mediator is the "what to do" and observer the "how to notify". **Facade** looks similar, but subsystem classes are unaware of a facade, whereas mediator colleagues explicitly depend on the mediator and communicate *only* through it.

Memento

also: *Snapshot*, *Token*

INTENT

Capture and externalise an object's internal state so it can be restored later — without violating the object's encapsulation.

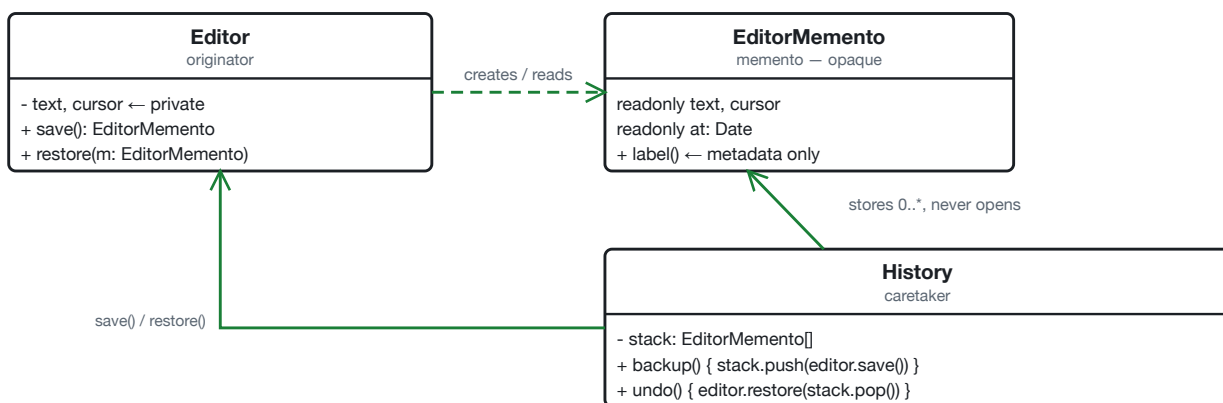
PROBLEM

You want undo in an editor. The obvious approach — an outside "history" object reading the editor's fields and stashing copies — requires making those fields public. Now every field is part of your public API: you can never refactor the editor's internals, and any code can corrupt it. The requirement (save state) directly conflicts with encapsulation.

SOLUTION

Let the **originator** produce its own snapshot. It returns a **memento** — an opaque object whose contents only the originator can read. A **caretaker** (the history stack) holds mementos without ever inspecting them, and hands one back to restore.

The "opaque" part is enforced differently per language: nested private classes in Java/C#, by convention with a narrow public interface in TypeScript, friend classes in C++.



Structure. Only the originator can build or read a memento; the caretaker treats it as an opaque token.

CODE

```
// The Memento: opaque to everyone but the Editor.
class EditorMemento {
  constructor(
    readonly text: string,
    readonly cursor: number,
    readonly at = new Date()) {}
  // Only metadata is public – the caretaker may show it in a menu.
  label() { return `${this.at.toISOString().slice(11,19)} – "${this.text.slice(0,12)}..."` }
}

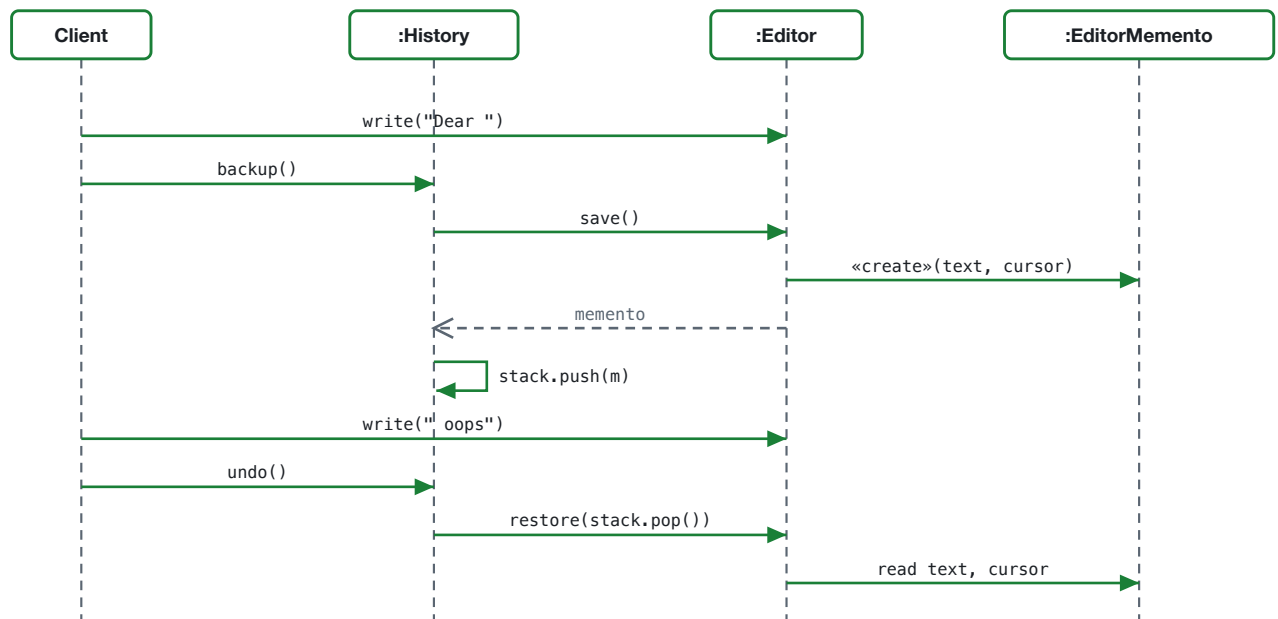
class Editor {
  private text = ""
  private cursor = 0

  write(s: string) { this.text += s; this.cursor = this.text.length }

  save(): EditorMemento { // only the Editor can build one
    return new EditorMemento(this.text, this.cursor)
  }
  restore(m: EditorMemento) { // only the Editor can read one
    this.text = m.text; this.cursor = m.cursor
  }
  show() { return `"${this.text}" ^${this.cursor}` }
}

// Caretaker: stores mementos, never opens them.
class History {
  private stack: EditorMemento[] = []
  constructor(private editor: Editor) {}
  backup() { this.stack.push(this.editor.save()) }
  undo() { const m = this.stack.pop(); if (m) this.editor.restore(m) }
  list() { return this.stack.map(m => m.label()) }
}

const ed = new Editor(), h = new History(ed)
ed.write("Dear "); h.backup()
ed.write("Alex,"); h.backup()
ed.write(" oops")
console.log(ed.show()) // "Dear Alex, oops" ^15
h.undo()
console.log(ed.show()) // "Dear Alex," ^10
```



Runtime. History moves mementos around without ever looking inside one.

MEMORY: THE WHOLE PRACTICAL PROBLEM

Full snapshots of a large document on every keystroke will exhaust memory. Real systems use: **COPY-ON-WRITE** or persistent (structurally shared) data structures, so unchanged parts are not duplicated; **INCREMENTAL** mementos storing only the delta; **CAPPED** history depth; or snapshots only at coarse checkpoints, replaying **COMMANDS** to reach intermediate states.

PROS

- Snapshots without breaking encapsulation.
- The caretaker's history code stays simple.
- Originator internals can be refactored freely.

CONS

- High memory use if snapshots are frequent or large.
- Caretakers must know when to discard mementos.
- Dynamic languages cannot truly enforce opacity.

In the wild: editor and IDE undo stacks; database savepoints and transaction rollback; game save files; Redux DevTools time-travel debugging; VM/container checkpoints. **Relations:** Pairs constantly with **Command** (command holds the memento needed to undo). Can be implemented via **Prototype** if cloning suffices. An **Iterator**'s position can be captured as a memento.

Observer

also: *Event-Subscriber, Listener, Publish-Subscribe*

INTENT

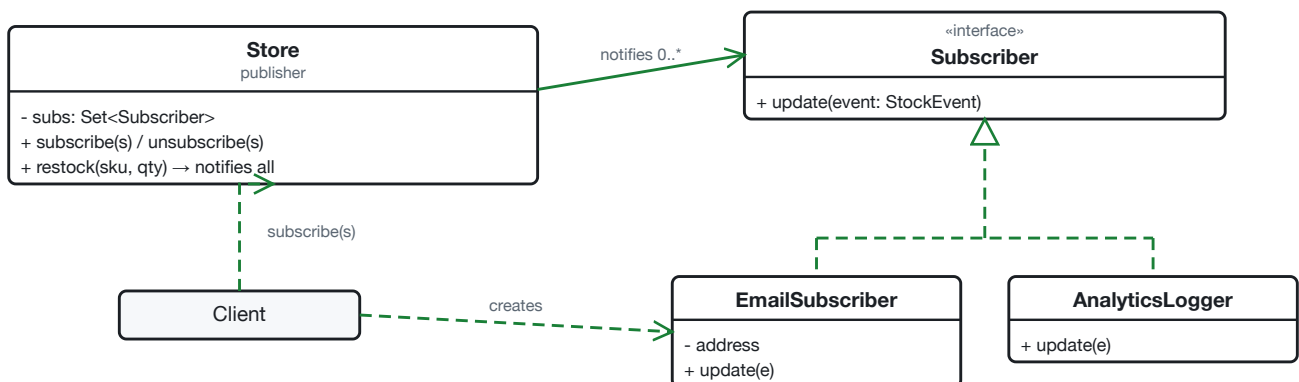
Define a one-to-many dependency so that when one object changes state, all its dependants are notified automatically.

PROBLEM

A store gets new stock. Some customers want an email; others want a push notification; others want nothing. Two bad options: the store emails every customer (spam, and the store now knows about email), or every customer polls the store daily (wasteful, and every customer now knows about the store). The store should not know who cares, and the interested parties should not have to ask repeatedly.

SOLUTION

Give the **publisher** a list of **subscriber** objects and methods to add and remove them. When the interesting event happens, the publisher loops the list and calls each subscriber's `update()`. The publisher depends only on the subscriber *interface*, so it never learns what any subscriber actually does.



Structure. The publisher depends only on the Subscriber interface; concrete subscribers come and go at run time.

CODE

```

type StockEvent = { sku: string; qty: number }

interface Subscriber { update(e: StockEvent): void }

class Store {
  private subs = new Set<Subscriber>()

  subscribe(s: Subscriber) { this.subs.add(s) }
  unsubscribe(s: Subscriber) { this.subs.delete(s) } // ← don't forget this

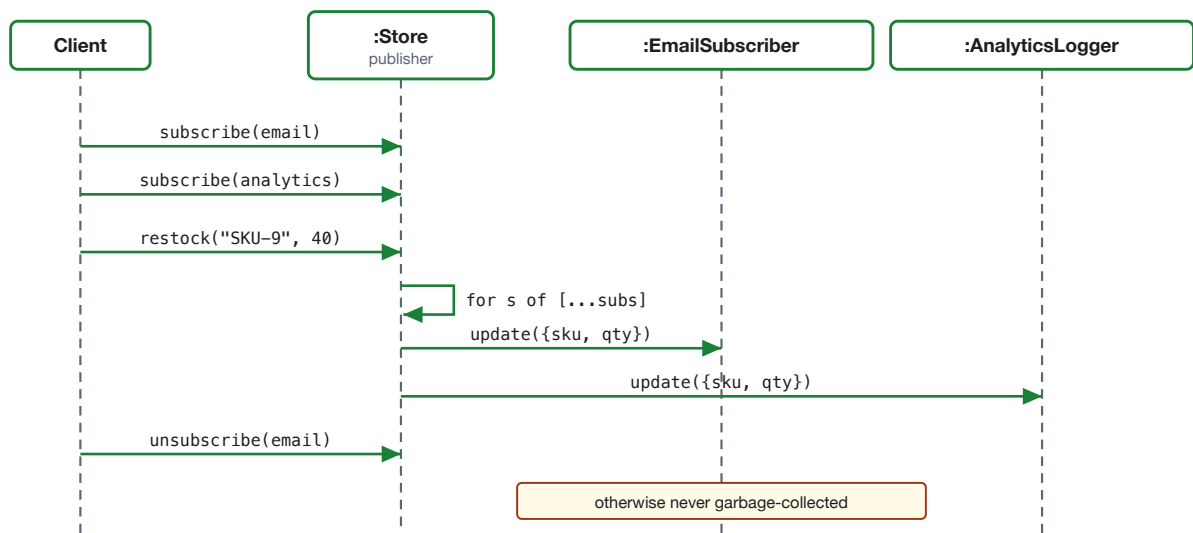
  restock(sku: string, qty: number) {
    // business logic first...
    const event = { sku, qty }
    // ...then notify. Copy the set: a subscriber may unsubscribe inside update().
    for (const s of [...this.subs]) {
      try { s.update(event) }
      catch (err) { console.error("subscriber failed:", err) } // isolate failures
    }
  }
}

class EmailSubscriber implements Subscriber {
  constructor(private address: string) {}
  update(e: StockEvent) { console.log(`mail-${this.address}: ${e.sku} x${e.qty}`) }
}

class AnalyticsLogger implements Subscriber {
  update(e: StockEvent) { console.log(`metric restock ${e.sku}`) }
}

const store = new Store()
const ana = new EmailSubscriber("ana@example.com")
store.subscribe(ana)
store.subscribe(new AnalyticsLogger())
store.restock("SKU-9", 40)
store.unsubscribe(ana) // otherwise ana is never garbage-collected

```



Runtime (push model). One state change fans out to every current subscriber.

THE THREE CLASSIC OBSERVER BUGS

1. **LAPSED LISTENERS.** A subscriber that forgets to unsubscribe is kept alive by the publisher's list forever — the most common memory leak in event-driven code. Use weak references, or have `subscribe` return a disposer function you must call.
2. **NOTIFICATION STORMS.** An update handler that changes the publisher triggers another notification, and so on — cycles and cascades. Guard with a re-entrancy flag or batch notifications.
3. **ORDER DEPENDENCE.** Subscribers are notified in some order; code that relies on that order is fragile. Treat notification order as undefined.

PUSH VS PULL

Push sends the changed data in the event (as above): fewer round trips, but the publisher must guess what subscribers want. **Pull** sends only "something changed" plus a reference to the publisher, and each subscriber queries what it needs: more flexible, more calls. Push is the common default; pull suits subscribers with very different needs.

PROS

- Add subscribers without changing the publisher (OCP).
- Relationships are established at run time.
- Publisher and subscribers are independently testable.

CONS

- Notification order is unspecified.
- Memory leaks from forgotten unsubscriptions.
- Cascading updates are hard to trace and can loop.
- An exception in one subscriber can break the rest unless isolated.

In the wild: DOM `addEventListener`; RxJS and every reactive library; Node's `EventEmitter`; MVC model→view updates; message brokers (a distributed observer with a durable broker in the middle); React's `useEffect` subscriptions. **Relations:** **Mediator** can be implemented with Observer for its notification channel; the distinction is that a mediator encodes rules while a publisher merely broadcasts. **Command** objects are often what a subscriber runs. Compare **Chain of Responsibility**: one handler acts, versus all subscribers act.

State

INTENT

Let an object alter its behaviour when its internal state changes, so that it appears to change its class.

PROBLEM

A document is `draft`, `moderation`, or `published`. Every method starts with a conditional on the current state:

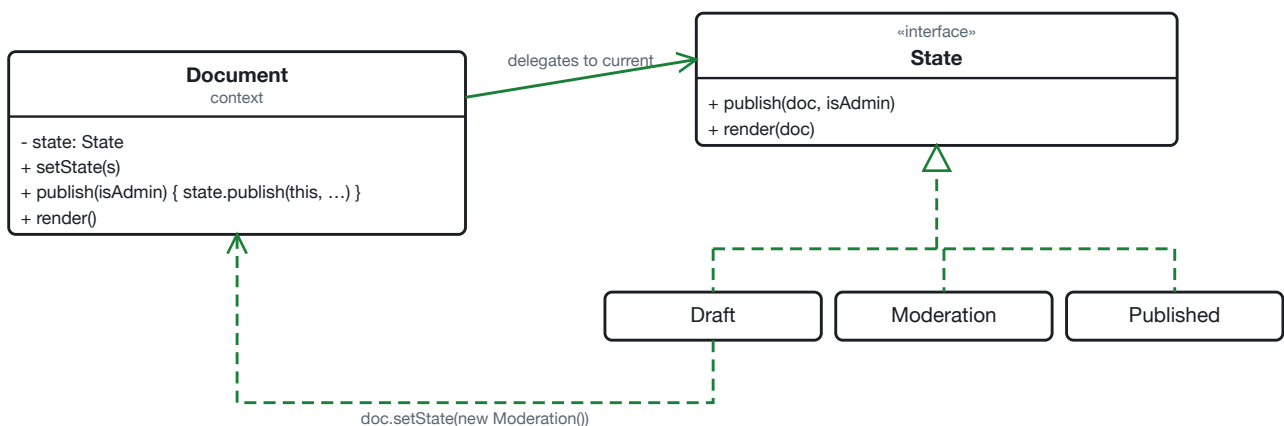
```
publish() {
  switch (this.state) {
    case "draft":    this.state = "moderation"; break
    case "moderation": if (user.isAdmin()) this.state = "published"; break
    case "published": break // nothing
  }
}
```

Adding an `archived` state means editing every method. The transition rules are scattered across the class instead of stated anywhere, and it is impossible to see the state machine by reading any single place.

SOLUTION

Create a class per state, all implementing a common state interface with one method per action. The **context** holds a reference to the current state object and delegates every call to it. Each state class implements only its own behaviour, and performs transitions by telling the context to switch to another state object.

The conditionals vanish: the object's *class* now encodes what its *state field* used to. Adding a state adds a class and changes nothing else.



Structure. Same shape as Strategy — but here the state objects trigger the switch to their successors.

CODE

```
interface State {
  readonly name: string
  publish(doc: Document, isAdmin: boolean): void
  render(doc: Document): string
}

class Document {
  private state: State = new Draft()
  constructor(public title: string) {}

  setState(s: State) { this.state = s } // states call this

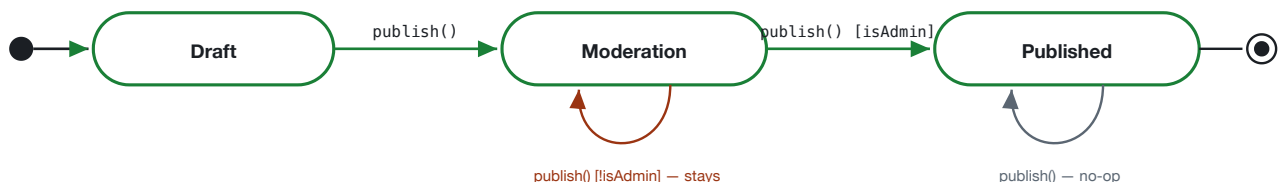
  // No conditionals anywhere in the context.
  publish(isAdmin = false) { this.state.publish(this, isAdmin) }
  render() { return this.state.render(this) }
  status() { return this.state.name }
}

class Draft implements State {
  readonly name = "draft"
  publish(doc: Document) { doc.setState(new Moderation()) }
  render(doc: Document) { return `[private draft] ${doc.title}` }
}

class Moderation implements State {
  readonly name = "moderation"
  publish(doc: Document, isAdmin: boolean) {
    if (isAdmin) doc.setState(new Published()) // rule lives with its state
  }
  render(doc: Document) { return `[awaiting review] ${doc.title}` }
}

class Published implements State {
  readonly name = "published"
  publish() { /* terminal state: nothing to do */ }
  render(doc: Document) { return `<article>${doc.title}</article>` }
}

const d = new Document("Patterns")
console.log(d.render()) // [private draft] Patterns
d.publish()
console.log(d.status()) // moderation
d.publish(false); console.log(d.status()) // moderation (blocked)
d.publish(true); console.log(d.render()) // <article>Patterns</article>
```



The state machine the classes implement. Each rounded node is one State class; each arrow is a `setState()` call inside a state.

WHERE DO TRANSITIONS BELONG?

Two schools. **STATES DECIDE** (above): each state knows its successors — cohesive, but states become coupled to each other. **THE CONTEXT DECIDES** via a transition table: states stay ignorant of each other and the whole machine is visible in one place, at the cost of a table to maintain. For complex machines with many states, prefer the table — or a dedicated state-machine library — since it makes illegal transitions checkable.

PROS

- Eliminates large state conditionals.
- Each state's behaviour and rules live in one class (SRP).
- New states don't touch existing ones (OCP).
- Stateless state objects can be shared as flyweights.

CONS

- Overkill for two states or rarely-changing machines.
- State objects need access to context internals — a coupling to manage.
- The overall machine is spread across many files.

In the wild: TCP connection states (LISTEN, ESTABLISHED, CLOSE_WAIT...); order and workflow engines; game character AI; media-player controls; lexer/parser modes. **Relations:** Structurally identical to **Strategy** (see the comparison in Part V) — the difference is that states know about and switch to each other, while strategies are unaware of one another and are chosen by the client. A refined **Bridge** if you squint. **Flyweight** applies when states hold no per-context data.

Strategy

INTENT

Define a family of interchangeable algorithms, encapsulate each one, and make them swappable at run time independently of the clients that use them.

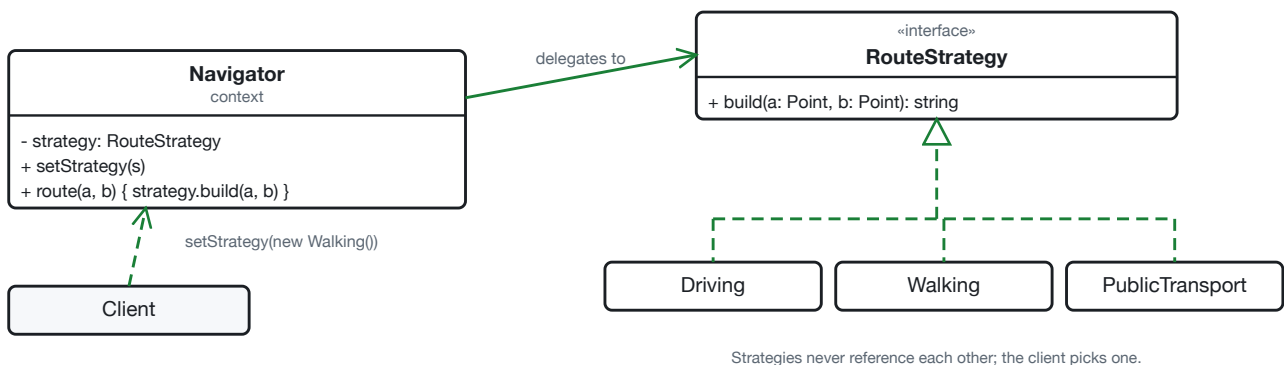
PROBLEM

A navigation app computes routes. It started with driving. Then walking. Then cycling, public transport, and scenic routes. Each addition doubled the size of one class, made merge conflicts constant, and meant that a bug in the cycling algorithm required redeploying the driving one. The class has many reasons to change — the clearest possible SRP violation.

SOLUTION

Extract each algorithm into its own class implementing a shared **strategy** interface. The **context** holds a reference to one strategy and delegates the work to it, without knowing which one it has. The client picks the strategy and passes it in.

Crucially, the context does not choose — that is what keeps it free of conditionals. The choice moves to whoever knows the user's intent.



Structure. The context holds one interchangeable strategy and knows nothing about which.

CODE

```
type Point = { x: number; y: number }

interface RouteStrategy {
  build(a: Point, b: Point): string
}

class Driving implements RouteStrategy {
  build(a: Point, b: Point) { return `motorway route, ${this.km(a,b)} km` }
  private km(a: Point, b: Point) { return Math.hypot(b.x-a.x, b.y-a.y).toFixed(1) }
}

class Walking implements RouteStrategy {
  build(a: Point, b: Point) { return "footpaths and crossings" }
}

class PublicTransport implements RouteStrategy {
  build(a: Point, b: Point) { return "bus 14 → metro M2 → walk 300 m" }
}

// Context: no conditionals, no algorithm knowledge.
class Navigator {
  constructor(private strategy: RouteStrategy) {}
  setStrategy(s: RouteStrategy) { this.strategy = s } // swap at run time
  route(a: Point, b: Point) {
    return "Route: " + this.strategy.build(a, b)
  }
}

const nav = new Navigator(new Driving())
console.log(nav.route({x:0,y:0}, {x:3,y:4})) // motorway route, 5.0 km
nav.setStrategy(new PublicTransport())
console.log(nav.route({x:0,y:0}, {x:3,y:4})) // bus 14 → metro M2 → ...

// In a language with first-class functions, a strategy is often just a function:
type RouteFn = (a: Point, b: Point) => string
const scenic: RouteFn = () => "the long way, past the lake"
// ...and Array.sort(comparator) is the strategy pattern you already use daily.
```

STRATEGY VS A PLAIN FUNCTION

If the algorithm is one operation with no state and no configuration, pass a function or lambda — the class buys you nothing. Prefer the full class form when the strategy needs its own fields (configuration, caches, injected dependencies), when it exposes more than one method, or when the set of strategies must be enumerable, named, and discoverable (e.g. a registry driven by a config file).

PROS

- Swap algorithms at run time.
- Isolates algorithm details from the context.
- Replaces conditionals with polymorphism.
- New strategies require no changes to the context (OCP).
- Each algorithm is testable in isolation.

CONS

- Clients must understand the strategies well enough to choose.
- Extra objects for what may be two branches.
- Strategies that need lots of context data lead to fat interfaces.

In the wild: comparators in `sort`; compression, hashing, and serialization format selection; pluggable auth schemes (Passport strategies); payment providers; caching eviction policies; ML model selection. **Relations:** **State** is the same structure with self-transitioning states. **Bridge** is the same structure applied to a whole design dimension. **Command** is the same structure applied

to "what to do" rather than "how to do it". **Template Method** solves the same problem with inheritance instead of composition, at class level rather than object level.

Template Method

INTENT

Define the skeleton of an algorithm in a base class, deferring some steps to subclasses, so subclasses can redefine those steps without changing the algorithm's structure.

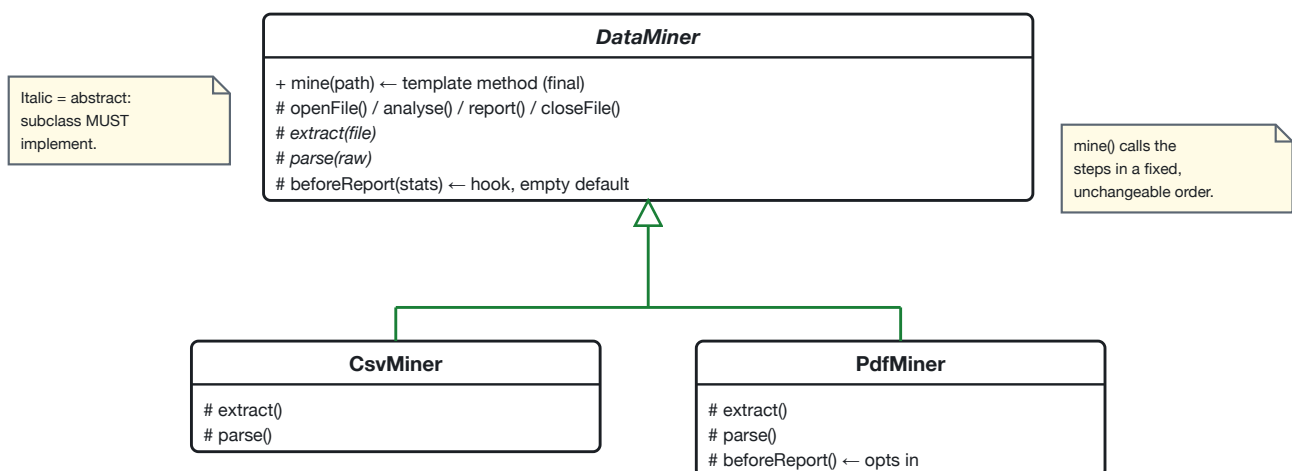
PROBLEM

You mine data from DOC, CSV, and PDF files. All three follow the same sequence: open the file, extract raw data, parse it, analyse it, produce a report, close the file. Only "extract" and "parse" actually differ. Written as three classes, the shared five-sixths is duplicated, and a fix to the reporting step must be applied three times — and inevitably will not be.

SOLUTION

Put the sequence in a single `final` method on an abstract base class — the **template method**. It calls a series of steps, some implemented in the base (shared behaviour), some declared abstract (the subclass must supply them), and some given a default empty body — these are **hooks**, optional extension points a subclass may override or ignore.

Subclasses fill in only what differs. The order of steps is fixed and unforgeable, which is sometimes exactly what you want (a subclass cannot forget to close the file).



Structure. One class hierarchy; the algorithm skeleton is inherited, the varying steps are overridden.

CODE

```
abstract class DataMiner {
  // THE TEMPLATE METHOD – the sequence is the contract. (TS has no `final`;
  // Java/C# would mark it final/sealed so subclasses cannot override it.)
  mine(path: string): string {
    const file = this.openFile(path)      // shared
    const raw  = this.extract(file)       // abstract – must override
    const data = this.parse(raw)          // abstract – must override
    const stats = this.analyse(data)      // shared
    this.beforeReport(stats)              // HOOK – optional
    const out = this.report(stats)        // shared
    this.closeFile(file)                  // shared, always runs
    return out
  }

  protected openFile(p: string) { return `<handle ${p}>` }
  protected closeFile(f: string) { /* release */ }
  protected analyse(rows: string[]) { return { count: rows.length } }
  protected report(s: { count: number }) { return `${s.count} records` }

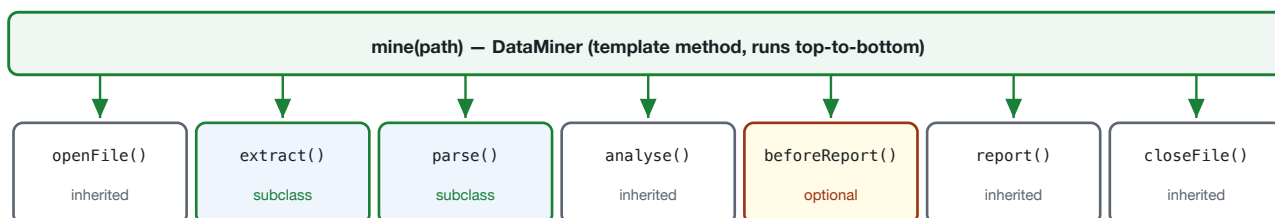
  // Hook: empty by default, subclasses may opt in.
  protected beforeReport(s: { count: number }) {}

  // The varying steps.
  protected abstract extract(file: string): string
  protected abstract parse(raw: string): string[]
}

class CsvMiner extends DataMiner {
  protected extract(f: string) { return "a,1\nb,2\nc,3" }
  protected parse(raw: string) { return raw.split("\n") }
}

class PdfMiner extends DataMiner {
  protected extract(f: string) { return "page1|page2" }
  protected parse(raw: string) { return raw.split("|") }
  // opts into the hook
  protected beforeReport(s: { count: number }) { console.log("OCR confidence check...") }
}

console.log(new CsvMiner().mine("sales.csv")) // 3 records
console.log(new PdfMiner().mine("scan.pdf"))  // OCR... then 2 records
```



grey = fixed in the base class · blue = subclass must supply · amber = hook a subclass may override

Runtime. Seven steps, fixed order; a CsvMiner supplies two of them and ignores the hook.

THE INHERITANCE TAX

Template Method is compile-time and single-inheritance. You cannot change a step at run time, a subclass cannot combine steps from two parents, and subclasses become tightly bound to the base class's protected surface — change a step's signature and every subclass breaks. Refused-bequest smells appear when a subclass must override a step to do nothing. When you feel that pressure, the answer is usually to convert the varying steps into injected **STRATEGY** objects, which is the composition-based version of the same idea.

PROS

- Removes duplication into a single base implementation.
- Subclasses override only what genuinely differs.
- Enforces invariant ordering — steps can't be skipped or reordered.
- Hooks give safe, documented extension points.

CONS

- Fixed skeleton limits what subclasses can express.
- Subclass–superclass coupling is tight and brittle.
- Steps are chosen at compile time only.
- Growing numbers of hooks make the base class hard to reason about.

In the wild: framework lifecycle methods (React class components, Android `Activity`, JUnit `setUp / tearDown`); `AbstractList` and friends; build-tool task phases; servlet `service()` dispatching to `doGet / doPost`. **Relations:** **Strategy** is its composition-based counterpart — Template Method works at class level via inheritance, Strategy at object level via delegation. **Factory Method** is a specialisation of Template Method whose step returns an object.

Visitor

INTENT

Represent an operation to be performed on the elements of an object structure, letting you define new operations without modifying the element classes.

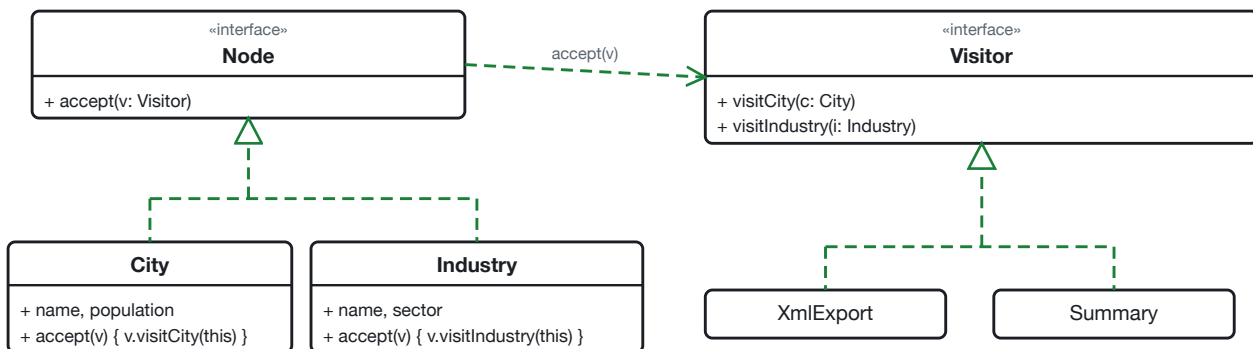
PROBLEM

You have a stable hierarchy of geographic nodes — `City`, `Industry`, `SightSeeing` — used across a large system. You need to export them to XML. Adding an `exportXml()` method to each node class means editing classes you were told not to touch, and it puts serialization logic inside domain objects. Next quarter you'll need JSON, then a distance calculator, then a validator — one new method on every class each time.

SOLUTION

Move each operation into a separate **visitor** class with one method per element type: `visitCity(c)`, `visitIndustry(i)`. Elements gain a single method, `accept(visitor)`, whose entire body is `visitor.visitX(this)`.

That one-line body is the trick, called **double dispatch**: the correct `accept` is chosen by the element's run-time type, and inside it the correct `visitX` is chosen because `this` is statically the element's own type. Two dynamic dispatches select the right (element, operation) pair without a single type check.



← new node type: edit every visitor ✗

new operation: one new class ✓ →

Structure. Operations live in visitors; elements only know how to accept one.

CODE

```
interface Visitor {
  visitCity(c: City): string
  visitIndustry(i: Industry): string
}

interface Node { accept(v: Visitor): string }

class City implements Node {
  constructor(readonly name: string, readonly population: number) {}
  accept(v: Visitor) { return v.visitCity(this) } // double dispatch
}

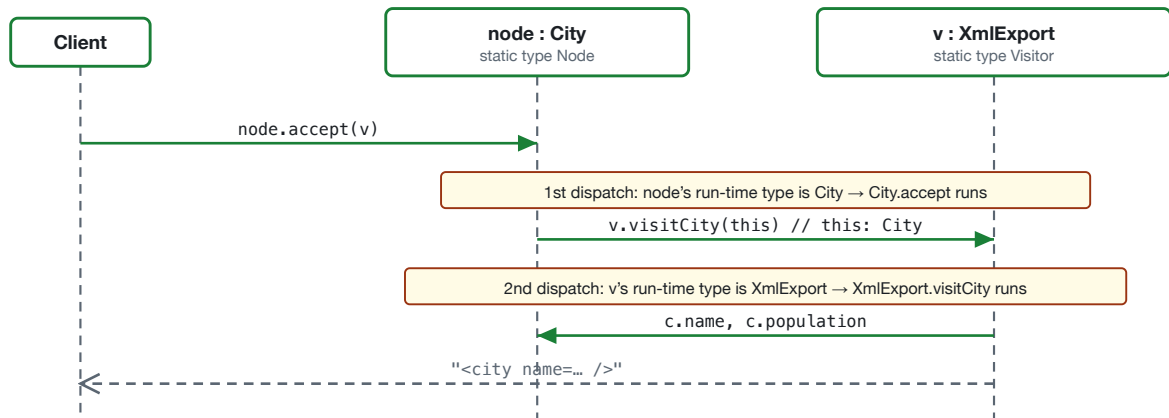
class Industry implements Node {
  constructor(readonly name: string, readonly sector: string) {}
  accept(v: Visitor) { return v.visitIndustry(this) }
}

// New operation #1 – zero changes to City or Industry.
class XmlExport implements Visitor {
  visitCity(c: City) { return `<city name="${c.name}" pop="${c.population}"/>` }
  visitIndustry(i: Industry) { return `<industry sector="${i.sector}"/>` }
}

// New operation #2 – again, zero changes to the node classes.
class Summary implements Visitor {
  visitCity(c: City) { return `${c.name}: ${c.population} people` }
  visitIndustry(i: Industry) { return `${i.name} (${i.sector})` }
}

const map: Node[] = [new City("Porto", 231_000), new Industry("Docks", "shipping")]

for (const n of map) console.log(n.accept(new XmlExport()))
for (const n of map) console.log(n.accept(new Summary()))
```



Double dispatch. Two virtual calls select the (element type, visitor type) pair with no type checks.

THE EXPRESSION PROBLEM — DECIDE BEFORE YOU COMMIT

Object-oriented code makes it cheap to add a new *type* (one class, all its methods) and expensive to add a new *operation* (edit every class). Visitor inverts this: cheap new operations, expensive new types. You cannot have both cheaply in a conventional OO language.

So: use Visitor only when the element hierarchy is **STABLE** and the set of operations is **GROWING**. If new element types arrive regularly, Visitor will be a permanent tax — every new node type breaks every visitor. In languages with pattern matching and sealed unions (Rust, Scala, Kotlin, modern Java/C#), a match expression gives you the same benefit with far less machinery and exhaustiveness checking from the compiler.

PROS

- New operations without touching element classes (OCP).
- Related behaviour is gathered in one visitor rather than scattered (SRP).
- A visitor can accumulate state across a whole traversal.
- Keeps unrelated concerns (export, validation, metrics) out of domain classes.

CONS

- Every new element type breaks every visitor.
- Visitors may need access to element internals, weakening encapsulation.
- Verbose and unfamiliar; high cognitive cost for readers.
- Pattern matching supersedes it in modern languages.

In the wild: compiler and linter passes over an AST (type-checking, optimisation, code generation are all visitors); document format converters; Java's `javax.lang.model.element.ElementVisitor` in annotation processors; Roslyn's

`CSharpSyntaxVisitor`. **Relations:** Nearly always applied over a **Composite** tree, usually driven by an **Iterator**. Can be seen as a very powerful **Command** executed against a whole structure.

Interpreter

INTENT

Given a language, define a class for each grammar rule and an interpreter that evaluates sentences in that language by walking a tree of those classes.

PROBLEM

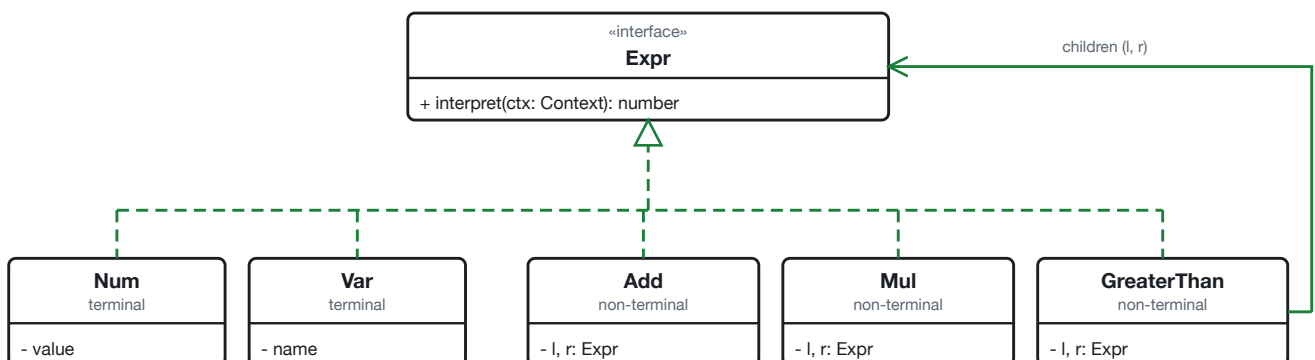
Your product needs user-defined rules: an alerting threshold, a discount condition, a search filter. Hard-coding each rule as a class does not scale, and the users are not programmers who can submit pull requests. You need to evaluate expressions supplied as data at run time — but `eval()` on user input is a code-execution vulnerability.

SOLUTION

Define a small grammar. Give every rule its own class implementing one `interpret(context)` method.

Terminal expressions (a number, a variable) return a value directly; **non-terminal expressions** (and, or, greater-than) hold child expressions and combine their results. A parsed sentence becomes a tree of these objects; evaluating it is one call to the root — which is exactly a **Composite** whose operation is "evaluate".

Because the tree only ever does what your expression classes allow, it is a safe sandbox: users cannot reach outside the grammar you defined.



Structure. One class per grammar rule; non-terminals hold child expressions — a Composite whose operation is "evaluate".

CODE

```
type Context = Record<string, number>

interface Expr { interpret(ctx: Context): number }

// --- terminal expressions ---
class Num implements Expr {
  constructor(private v: number) {}
  interpret() { return this.v }
}
class Var implements Expr {
  constructor(private name: string) {}
  interpret(ctx: Context) {
    if (!(this.name in ctx)) throw new Error(`unknown var ${this.name}`)
    return ctx[this.name]
  }
}

// --- non-terminal expressions: hold children ---
class Add implements Expr {
  constructor(private l: Expr, private r: Expr) {}
  interpret(c: Context) { return this.l.interpret(c) + this.r.interpret(c) }
}
class Mul implements Expr {
  constructor(private l: Expr, private r: Expr) {}
  interpret(c: Context) { return this.l.interpret(c) * this.r.interpret(c) }
}
class GreaterThan implements Expr {
  constructor(private l: Expr, private r: Expr) {}
  interpret(c: Context) { return this.l.interpret(c) > this.r.interpret(c) ? 1 : 0 }
}

// "cart * 0.9 > threshold" - built by a parser from user input
const rule: Expr = new GreaterThan(
  new Mul(new Var("cart"), new Num(0.9)),
  new Var("threshold"))

console.log(rule.interpret({ cart: 120, threshold: 100 })) // 1 (108 > 100)
console.log(rule.interpret({ cart: 100, threshold: 100 })) // 0 (90 > 100 false)
```



The expression tree for the code example, with each node's interpret() result. Rounded = terminal.

SCOPE LIMIT

Interpreter is only viable for **SMALL, STABLE GRAMMARS**. One class per rule means a real programming language would need hundreds, and a tree-walking interpreter is slow. For anything beyond a simple expression language, use a parser generator (ANTLR, tree-sitter), an existing embeddable language (Lua, Starlark, CEL, JSONLogic), or a rules engine. The pattern's lasting value is conceptual: it is how you get a safe, sandboxed DSL without `eval`.

Note also that Interpreter covers only *evaluation*. Turning text into the tree — parsing — is a separate problem the pattern says nothing about.

PROS

- Grammar rules map one-to-one onto classes — easy to read and extend.
- New rules are new classes, no edits elsewhere.
- Safe evaluation of user-supplied logic.
- Trees can be inspected, optimised, serialised, and explained.

CONS

- Class explosion for anything but a tiny grammar.
- Tree-walking evaluation is slow versus compiled approaches.
- Says nothing about parsing.
- Rarely the right tool today — libraries usually win.

In the wild: regular-expression engines; SQL `WHERE`-clause evaluators; spreadsheet formula engines; feature-flag targeting rules; JSONLogic and CEL; ORM query expression trees. **Relations:** The expression tree *is* a **Composite**. Operations over it are often implemented as **Visitors** (optimise, pretty-print, compile). Traversal uses **Iterator**; shared terminals can be **Flyweights**.

Part V — Applying Patterns

1 • Choosing a pattern from a symptom

Patterns are answers to problems, so start from the problem. Find the symptom you actually have in the left column.

SYMPTOM IN YOUR CODE	CANDIDATE	WHY
Direct <code>new Concrete()</code> scattered through business logic	Factory Method	Moves the class choice to one overridable place
Objects from different families get mixed incorrectly	Abstract Factory	Holding one factory makes mismatch impossible
A constructor with many optional parameters	Builder	Named steps instead of positional flags
Expensive setup repeated for near-identical objects	Prototype	Copy a configured exemplar
A shared resource must exist exactly once	Singleton — or DI	Prefer one instance at the composition root
A third-party interface doesn't match yours	Adapter	Translate at the boundary
Class count = variants × platforms	Bridge	Split the two axes into separate hierarchies
Client code recurses and type-checks over a tree	Composite	Push recursion into the structure
Every combination of optional behaviours needs a subclass	Decorator	Stack wrappers at run time
Callers must orchestrate ten objects in a fixed order	Facade	Name the workflow, hide the wiring
Memory blown by duplicated identical data	Flyweight	Share intrinsic state through a pool
Need lazy loading, caching, or access checks transparently	Proxy	Same interface, controlled access
A method is a long ladder of pre-checks	Chain of Responsibility	One handler per check, ordered at run time
Need undo, queuing, retry, or an audit log of actions	Command	The call becomes an object you can store
Traversal logic leaks into client loops	Iterator	Traversal becomes its own object
Components hold references to each other in a mesh	Mediator	Route everything through a hub
Need snapshot/restore without exposing fields	Memento	The object snapshots itself
Many parties must react to one object's changes	Observer	Publisher knows only an interface
Every method switches on a <code>status</code> field	State	One class per state; conditionals disappear
One class holds many variants of the same computation	Strategy	One class per algorithm, injected

Several classes duplicate a sequence, differing in two steps	Template Method	Skeleton in the base, steps in subclasses
New operations keep getting added to a stable hierarchy	Visitor	Operations become classes, elements untouched
Users must supply logic as data at run time	Interpreter	A grammar tree you can evaluate safely

2 • Look-alike pairs

The patterns most often confused are the ones with identical UML and different intent. These four comparisons resolve most of the confusion.

Strategy vs State

	STRATEGY	STATE
What varies	An algorithm — <i>how</i> to do a job	Behaviour across a lifecycle — <i>what</i> the object currently is
Who chooses	The client, from outside	The states themselves, from inside
Do the parts know each other?	No — strategies are mutually ignorant	Yes — a state names its successors
Changes during a lifetime?	Rarely; often set once	Constantly — that's the point
Example	Route by car / on foot	Order: pending → paid → shipped

Test: if the objects transition into one another, it's State. If an outsider picks one and it stays, it's Strategy.

Proxy vs Decorator vs Adapter vs Facade

PATTERN	INTERFACE	PURPOSE	WHO ASSEMBLES IT
Adapter	Changed	Make incompatible things fit	Client, deliberately
Decorator	Unchanged	Add behaviour; always delegates	Client, stackable
Proxy	Unchanged	Control access; may not delegate at all	Often the proxy itself / the framework
Facade	New, simpler	Hide a whole subsystem	Not substitutable for the subsystem

Test: can the wrapper skip calling the wrapped object (cache hit, denied access)? Then it's a Proxy. Does it change the method signatures? Adapter. Neither, and stackable? Decorator.

Template Method vs Strategy

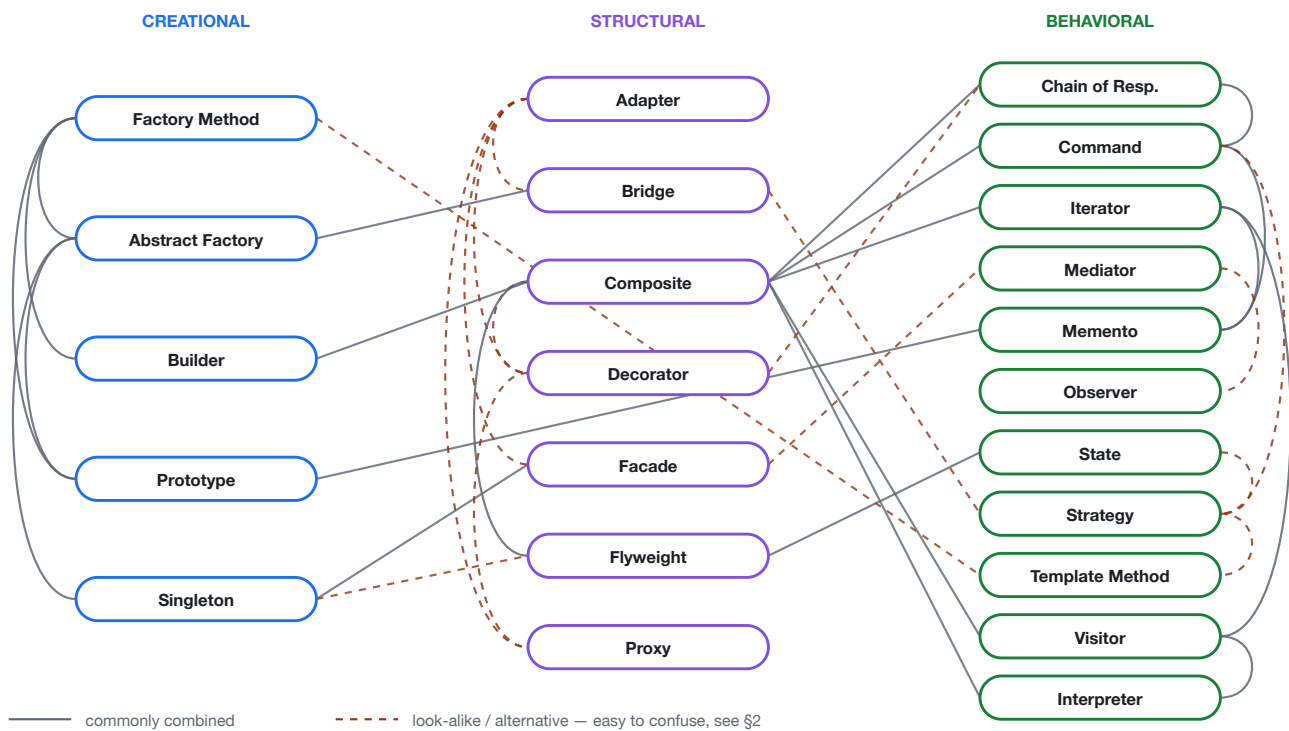
Same goal — vary part of an algorithm — reached two ways. Template Method uses **inheritance**: compile-time, one variant per subclass, base class controls the sequence. Strategy uses **composition**: run-time swappable, combinable, testable in isolation. Choose Template Method when the sequence itself is the invariant worth enforcing; choose Strategy when the variation must be selectable at run time or must be reusable outside this hierarchy.

Mediator vs Observer vs Facade

	COUPLING DIRECTION	KNOWS THE RULES?
Observer	Publisher → many subscriber interfaces; subscribers know the publisher	No — it just broadcasts
Mediator	Every colleague → the mediator, and back	Yes — all interaction logic lives there
Facade	Client → facade → subsystem; the subsystem is unaware	No — it only orchestrates a workflow

3 · Which patterns combine

Patterns are rarely used alone. These pairings are so common they are almost idioms.



Relationship map. Solid lines are frequent combinations; dashed lines join patterns with near-identical structure but different intent.

COMBINATION	TYPICAL USE
Composite + Visitor	Compiler passes over an AST; document tree transformations
Composite + Iterator	Walking a file system or DOM subtree
Composite + Builder	Assembling a deep tree with readable code
Command + Memento	Undo/redo: the command stores the snapshot it needs to reverse
Command + Composite	Macro commands — a command made of commands
Mediator + Observer	Mediator holds the rules; observer delivers the notifications
Abstract Factory + Bridge	Creating matched abstraction/implementor pairs per platform
Flyweight + Composite	Sharing identical leaf nodes in a huge tree
Flyweight + State	Stateless state objects shared across all contexts
Singleton + Facade / Factory	One well-known entry point to a subsystem
Strategy + Factory Method	Choosing a strategy by key from configuration
Proxy + Decorator	An auth proxy wrapping a caching proxy wrapping the real service
Interpreter + Composite + Visitor	Expression tree, evaluated and optimised by separate visitors

4 • Misuse and over-engineering

The most expensive pattern mistake is not choosing the wrong one — it is using one where none was needed. Signals that you have over-applied:

- **Abstraction with one implementation.** An interface, a factory, and a strategy, all with exactly one concrete class and no second one on the roadmap. You paid the indirection cost for no flexibility. Add the abstraction when the second case arrives.
- **Pattern names in class names everywhere.** `OrderStrategyFactoryProviderImpl`. Names should describe the domain; if the pattern name is doing the describing, the class probably has no domain meaning.
- **Indirection you cannot trace.** If finding where work actually happens takes four jump-to-definitions, the design is costing more than it saves.
- **Patterns for language gaps you don't have.** Singleton where a module constant works; Iterator where a generator works; Strategy/Command where a function works; Visitor where pattern matching works.
- **Speculative generality.** Structures built for requirements nobody has asked for. The requirement usually arrives in a different shape than you guessed, and the structure has to be demolished anyway.

THE WORKING RULE

Write the simplest code that solves today's problem. When a change becomes painful, identify the axis of change causing the pain, and apply the pattern that encapsulates *that* axis. Patterns are best arrived at by refactoring, not by up-front design — that is why Fowler's *Refactoring* and the GoF catalogue are companion books rather than rivals.

PATTERNS WHOSE MODERN REPLACEMENT IS USUALLY BETTER

PATTERN	PREFER INSTEAD, WHEN AVAILABLE
Singleton	Dependency injection with a singleton-scoped registration; a module-level constant
Iterator	The language's iteration protocol; generators; lazy sequence libraries
Strategy / Command	First-class functions and closures, when there's no state or config to hold
Visitor	Exhaustive pattern matching over a sealed/algebraic type
Interpreter	A parser generator or an embeddable expression language (CEL, JSONLogic, Starlark)
Prototype	<code>structuredClone</code> , copy constructors, or immutable-with- <code>copy()</code> value types
Observer	Reactive streams or a framework's built-in signals — mainly for the lifecycle management they add

5 • Self-test bank

Cover the answer key on the following pages. Aim to answer in one or two sentences — if you cannot, re-read that pattern's entry rather than the whole chapter.

FOUNDATIONS (1–8)

1. How does a design pattern differ from an algorithm?
2. Name the three pattern categories and the concern each addresses.
3. State the "encapsulate what varies" principle and name two patterns that apply it.
4. Why does composition scale better than inheritance when two features are independent?
5. Give a concrete example of a Liskov Substitution violation.
6. What does the Dependency Inversion Principle actually invert?
7. What is the strongest argument *against* learning design patterns?
8. Which principle is violated by a class that both formats a report and emails it?

CREATIONAL (9–20)

9. What exactly does a subclass override in Factory Method?
10. Why is a `switch`-based "simple factory" not the GoF Factory Method?
11. What does Abstract Factory guarantee that separate factory methods do not?
12. Which axis is Abstract Factory *closed* against extending cheaply, and why?
13. What two problems does Builder solve at once?
14. What does a Builder's director add, and what capability do you lose without one?
15. Why do most real-world "builders" not implement the full GoF pattern?
16. Why must the copying logic in Prototype live inside the class being copied?
17. Give two ways a naive deep clone can go wrong.
18. Name the two separate needs Singleton bundles, and which is legitimate.
19. Why is lazy Singleton initialisation unsafe in a multi-threaded program, and what's the simplest fix?
20. How does dependency injection give you Singleton's benefit without its costs?

STRUCTURAL (21–34)

21. Four structural patterns are wrappers. Distinguish them by the interface each exposes.
22. When would you write an Adapter rather than just editing the service class?
23. What does Bridge do to a class hierarchy that grows as $m \times n$?
24. Bridge and Strategy have the same UML. What actually differs?
25. What is the one-line body of a Composite's operation, conceptually?
26. Describe the trade-off in putting `add()` / `remove()` on Composite's component interface.
27. How does Decorator avoid the 2ⁿ subclass explosion?
28. Give a Decorator bug that only appears when wrappers are applied in the wrong order.
29. In `new BufferedReader(new InputStreamReader(new FileInputStream(f)))`, which wrapper is a Decorator and which is really an Adapter? Why?
30. What is a Facade's characteristic failure mode, and two ways to prevent it?
31. Define intrinsic and extrinsic state in Flyweight.
32. Why must flyweights be immutable?
33. Name four kinds of Proxy and what each controls.
34. Give the practical one-question test that distinguishes Proxy from Decorator.

BEHAVIORAL (35–52)

35. What does a Chain of Responsibility handler do that a Decorator never does?
36. Distinguish "pure" CoR from "pipeline" CoR and give an example of each.
37. What does Command turn a method call into, and which four capabilities follow?
38. Compare snapshot-based and inverse-operation-based undo.
39. Why is Command the right seam between a UI layer and business logic?
40. Why can one collection expose several iterators, and why does that matter for trees?
41. Which language features have largely absorbed Iterator?
42. How does Mediator change coupling from n^2 to n ?
43. What is the Mediator's occupational hazard, and two mitigations?
44. How does Memento capture state without breaking encapsulation?
45. Name three techniques for keeping Memento's memory use bounded.
46. Explain the "lapsed listener" leak and two fixes.
47. Contrast push and pull notification in Observer.
48. Why should you never rely on Observer notification order?
49. In State, what replaces the `switch` statements?
50. Compare "states decide transitions" against "the context decides".
51. When is a plain function a better Strategy than a Strategy class?
52. What invariant does Template Method enforce that Strategy cannot?

VISITOR, INTERPRETER, AND SYNTHESIS (53–60)

53. Explain double dispatch in Visitor and why `accept` is needed at all.
54. State the expression problem and say which side Visitor optimises for.
55. When does Visitor become a permanent tax rather than a benefit?
56. What modern language feature supersedes Visitor, and why is it better?
57. Why is Interpreter a Composite in disguise?
58. Why is Interpreter safer than `eval()` for user-supplied rules?
59. Which three patterns commonly combine to build and process an expression tree?
60. Give three signals that you have over-applied patterns to a codebase.

Answer key

1. An algorithm prescribes exact steps toward a result; a pattern prescribes a structural shape — which responsibilities live where. Two correct implementations of a pattern can share no code.
2. Creational — how objects are made. Structural — how objects are composed. Behavioral — how responsibility and communication are assigned.
3. Isolate the part most likely to change behind a stable boundary. Strategy (varying algorithm), State (varying mode), Factory Method (varying class), Decorator (varying extras).
4. Inheritance needs a class per *combination* ($m \times n$); composition needs a class per *part* ($m + n$), and the combination is chosen at run time.
5. `Square extends Rectangle`: code doing `setWidth(5); setHeight(4); assert area()==20` breaks for a `Square`, because `Square` strengthens the invariant its supertype promised.
6. The direction of the dependency between policy and detail. Instead of high-level code depending on low-level code, both depend on an abstraction the high-level code owns.
7. Several patterns exist only to work around missing language features; applying them in a language that has closures, generators, modules, or pattern matching adds ceremony with no benefit. They also invite speculative generality.
8. Single Responsibility — it has two unrelated reasons to change (layout rules, mail transport).
9. Only the factory method itself — the one method that returns a product. All business logic stays in the base class and works through the product interface.
10. There is no subclassing and therefore no open/closed benefit: adding a product still means editing the switch. It's a useful convenience, not the pattern.
11. That every product a client obtains belongs to the same family — mixing a `MacButton` with a `WinCheckbox` becomes unrepresentable, because the client holds exactly one factory.
12. Adding a new product *kind*. That requires a new method on the factory interface, which breaks every existing concrete factory. It is closed against new families, open against new kinds only at high cost.
13. Constructors with many optional parameters (or a subclass per combination), and the need for one construction sequence to yield different representations.
14. Reusable named recipes (`makeVilla`). Without it you lose the ability to run the same sequence against a different builder to get a different product type.
15. They implement only the fluent-interface half: one builder, one product, no director. That covers the readability need, which is what most people actually wanted.
16. Only code inside the class can read its private fields, and only it can name its own concrete type — so external copying is both impossible and coupling.
17. Reference cycles cause infinite recursion (fix: a visited map from original to copy); and serialization shortcuts like `JSON.parse(JSON.stringify(x))` silently drop Dates, Maps, Sets, functions, and class identity. Also: copying ids/timestamps that should be fresh.
18. (a) Restricting instantiation to one instance — legitimate when a second instance would be incorrect. (b) Providing global access — that's a global variable in disguise.
19. Two threads can both observe `null` and both construct. Simplest fix: eager static final initialisation, which the language guarantees runs once. Otherwise a holder class, or double-checked locking with a `volatile` field.
20. You create exactly one instance at the composition root and inject it. You keep single-instance behaviour but gain visible dependencies, substitutability in tests, and no global state.
21. Adapter exposes a *different* interface; Decorator and Proxy expose the *same* one; Facade exposes a *new, simpler* one over many objects.
22. When you don't own the service (third-party or legacy), when other clients depend on its current interface, or when you deliberately want a swap-out seam at the boundary.
23. It converts it to $m + n$ by extracting one axis into a separate implementor hierarchy held by reference.
24. Granularity and intent. Strategy swaps one algorithm inside a fixed class, often per call; Bridge splits an entire design dimension, is usually fixed at construction, and expects both sides to grow into hierarchies.
25. "Delegate to each child, then combine the results." The leaf does the real work; the composite aggregates.
26. Full uniformity for clients, but leaves must implement them by throwing — a Liskov violation. Restricting them to Composite keeps types honest but forces downcasts when building trees.
27. Each behaviour is a wrapper implementing the same interface, so behaviours stack linearly at run time: n classes cover all 2^n combinations.
28. E.g. compress-then-encrypt versus encrypt-then-compress: the second barely compresses. Or a caching decorator placed outside an auth decorator, so cached data is served to unauthorised callers.
29. `BufferedReader` is a Decorator: it wraps a `Reader` and exposes a `Reader`, adding buffering and `readLine()`. `InputStreamReader` is an Adapter: it wraps an `InputStream` (bytes) but exposes a `Reader` (chars) — the interface changes, which is the defining property of Adapter.
30. Becoming a god object that everything depends on. Prevent by keeping facades task-scoped, allowing several small ones per subsystem, and keeping business logic out of them.
31. Intrinsic: shared, immutable, identical across many objects (a texture, a glyph). Extrinsic: unique per object (position, size),

stored outside or passed as arguments.

32. Because one instance is shared by many contexts, any mutation would silently affect all of them. Immutability also makes them thread-safe for free.

33. Virtual (lazy creation), Remote (cross-process marshalling), Protection (permission checks), Caching (answers without touching the subject), plus logging/metering and smart references.

34. "Can the wrapper decide *not* to call the real object at all?" If yes — cache hit, denied permission — it's a Proxy. A Decorator always delegates.

35. Decline to pass the request on, ending the chain. A Decorator always delegates inward.

36. Pure: exactly one handler processes it and the chain stops (DOM event bubbling, exception handlers). Pipeline: every handler contributes and passes along, with optional early exit (HTTP middleware).

37. Into an object holding its receiver and arguments. That makes it storable, so you get queuing/scheduling, logging/audit, undo/redo, and composition into macro commands.

38. Snapshots are simple and always correct but memory-heavy; inverse operations are compact but must be exactly right for every command and every edge case. Editors favour inverses; databases log both.

39. The UI element depends only on the `Command` interface, and the command knows the receiver — so three different triggers share one implementation and the UI never references business classes.

40. Each iterator holds its own position and traversal rules, so several can run concurrently. For trees this matters because depth-first, breadth-first, and in-order are all equally valid traversals of the same structure.

41. Language iteration protocols (`for...of` / `Symbol.iterator` , `__iter__` , `IEnumerable` , `Iterable`) and especially generators, which give laziness in a few lines.

42. Each component keeps exactly one reference — to the mediator — instead of references to every component it interacts with, so links grow linearly with component count.

43. Becoming a god object, since all the removed coupling now lives in it. Mitigate by using one mediator per cohesive interaction cluster and by pushing genuine domain rules out into domain objects.

44. The originator produces the snapshot itself and returns an opaque memento; the caretaker stores it without reading it and hands it back to restore. No field ever becomes public.

45. Structural sharing / copy-on-write persistent data structures; incremental deltas rather than full snapshots; a capped history depth or coarse checkpoints with command replay in between.

46. A subscriber that never unsubscribes is kept alive by the publisher's list forever. Fixes: weak references, or have `subscribe` return a disposer the caller must invoke (and tie it to the subscriber's lifecycle).

47. Push sends the changed data with the event — fewer calls, but the publisher must guess what's wanted. Pull sends only "something changed" plus a reference, and each subscriber queries what it needs — more flexible, more calls.

48. The publisher's iteration order is an implementation detail that can change (set ordering, re-subscription, concurrency). Code depending on it breaks unpredictably.

49. Polymorphic dispatch on the current state object. The context simply delegates; the state's class encodes what the state field used to.

50. States deciding is cohesive but couples states to each other. A context-owned transition table keeps states mutually ignorant and makes the whole machine visible and checkable in one place — better for complex machines.

51. When it's a single stateless operation with no configuration, dependencies, or extra methods, and the set doesn't need to be enumerable or named.

52. The order and presence of steps. A subclass cannot skip or reorder them — a strategy consumer has no such guarantee.

53. `element.accept(v)` dispatches on the element's run-time type; inside `accept` , `v.visitX(this)` dispatches on the visitor's run-time type while `this` is statically the concrete element type. Single dispatch alone cannot select on two types at once, which is why `accept` exists.

54. In conventional OO you can cheaply add new types *or* new operations, not both. Visitor optimises for new operations, at the cost of making new element types expensive.

55. When the element hierarchy keeps growing: every new element type forces an edit to every existing visitor.

56. Exhaustive pattern matching over sealed/algebraic types — far less machinery, and the compiler proves you handled every case, which `accept/visit` cannot.

57. Terminal expressions are leaves and non-terminals hold child expressions; evaluating the root recursively delegates to children and combines results — exactly Composite, with "interpret" as the operation.

58. The tree can only do what your expression classes implement, so user input cannot reach outside the grammar. `eval()` hands the user your entire runtime.

59. Interpreter (the rule classes), Composite (the tree), and Visitor (passes such as optimise, pretty-print, compile) — usually with Iterator for traversal.

60. Abstractions with exactly one implementation; pattern names doing the work of domain names; and needing several jump-to-definitions to find where anything actually happens. Add speculative structures built for requirements nobody asked for.

WHERE TO GO NEXT

- *Design Patterns: Elements of Reusable Object-Oriented Software* — Gamma, Helm, Johnson, Vlissides (1994). The original catalogue; dense, C++-flavoured, still the authority on intent.
- *Refactoring* — Fowler. The companion volume: how you actually *arrive* at patterns, by naming code smells and the mechanical steps that resolve them.
- *Patterns of Enterprise Application Architecture* — Fowler. Repository, Unit of Work, Data Mapper, and the rest of the layer above this catalogue.
- *Domain-Driven Design* — Evans. Where the object model itself comes from, which is the question patterns don't answer.
- Read a framework's source with this guide open. Pick one you use daily and name the patterns you find — that exercise teaches more than any second reading.

Written as an original study guide. Pattern names and the three-category catalogue structure are the standard vocabulary established by Gamma, Helm, Johnson & Vlissides (1994); all prose, diagrams, and code in this document are newly written.